A Hyper-Personalized Recommendation Engine for OTT Platforms

Two-stage retrieval and ranking, bandit-driven exposure for new releases, and a serverless deployment on AWS

A project report submitted in partial fulfilment of the requirements for the degree of

Master of Business Administration (Business Analytics)

Semester 4

Submitted by

Sudarshan

Declaration

I, Sudarshan, declare that this project report, "A Hyper-Personalized Recommendation Engine for OTT Platforms", submitted in partial fulfilment of the requirements for the degree of Master of Business Administration (Business Analytics), is a record of my own work carried out under the guidance of my project guide.

The data, methods and results in this report are reported as they were obtained. Sources are acknowledged in the text and in the reference list. The use of an AI coding and writing assistant is described in Section 7.6 and Appendix H.

This report has not been submitted, in full or in part, for any other degree or diploma.

Sudarshan

Acknowledgements

I would like to thank my project guide for the project brief and for insisting that TensorFlow be part of it. That requirement shaped the design in ways I did not expect, mostly for the better.

I am grateful to my institute and the faculty of the Business Analytics programme for the foundation in statistics and machine learning that this project stands on.

This work would not have been possible without public data. Thanks to the GroupLens research lab at the University of Minnesota for the MovieLens datasets, and to The Movie Database (TMDB) for its API. This product uses the TMDB API but is not endorsed or certified by TMDB.

Finally, thanks to my family and friends for their patience during a month of late nights, long training runs and a laptop fan that never quite stopped.

Executive Summary

The problem. India has about 601 million OTT viewers and 148 million paid subscriptions (afaqs!, 2025). Each platform offers thousands of titles, but a viewer decides within seconds whether the home screen has anything worth watching. Recommendations decide what fills that screen. They also have to give new releases a fair chance, use the viewing context where it helps, and run fast and cheaply.

What I built. Following the project brief, I built a complete recommendation engine with four parts and deployed it on AWS:

1. **Candidate generation:** a two-tower model in TensorFlow narrows 33,392 movies to 200 candidates per user, and a Sentence-BERT content lane adds 30 new releases that no rating-based model can see. The vectors were also loaded into a Zilliz (Milvus) vector database.
2. **Ranking:** an XGBoost ranker with 37 inputs, including an NCF score from TensorFlow, the time of day and the user's session, orders the candidates.
3. **New releases:** a UCB bandit fills 2 of the 20 slots on screen with new titles.
4. **Serving:** a numpy-only AWS Lambda function with DynamoDB, in precomputed and live vector-search modes.

The data was MovieLens 25M (a 20,000-user sample, 3.1 million ratings) with TMDB movie descriptions. Device is not in the data, so every device value is SIMULATED, and the bandit was evaluated in a SIMULATION, NOT AN A/B TEST.

How I evaluated it. I split the data by time, compared every model with simple baselines, and wrote down my predictions before every major run. The test set was locked in code and opened exactly once, for five pre-registered claims.

What I found. All five headline results pointed in the predicted direction:

- **The learned ranker works.** It improved the quality of the top 10 (NDCG@10) by 18% over the candidate generator's own order (+0.0245, 95% interval +0.0229 to +0.0261).
- **Context barely helps on this data.** Time and session added under 1% (+0.0013). Time of day on its own made results slightly worse.
- **The two-tower model is the better retriever.** It found 58.5% of users' future likes in its top 200, against 53.8% for item-based nearest neighbors.

- **Tuning the bandit mattered more than the algorithm.** In the simulation, UCB with a tuned constant captured 94% of the possible gain, beating the textbook UCB1 by 42% and Thompson sampling by 4%.
- **The content lane is the only path for brand-new titles.** Rating-based models scored zero on them.

The deployed service answered warm requests in 91 ms (precomputed) and 122 ms (live) of server time, with zero errors in 400 timed requests. Most of that time is the ranker, not the database. Cold starts take about 2 seconds. Live vector search returned about 92% of the exact candidates and 93% of the same top-18 titles. Serving would cost roughly $3 per million requests at published AWS prices.

What I recommend. For an Indian OTT platform:

1. Invest first in a two-stage system with a learned ranker; it gave the biggest, most reliable gain from data the platform already has.
2. Treat context as a hypothesis: log real local time and device, and keep them only if an online test shows a gain.
3. Reserve the last two slots of the first screen for new releases, filled by a tuned bandit.
4. Serve precomputed recommendations by default, use live search where freshness matters, and fix the ranker's speed before scaling.
5. Confirm everything with a two-week A/B test before rollout, because all results here are offline.

Limitations. MovieLens ratings are a stand-in for viewing, the audience is not India's, device is simulated, and the bandit is simulated. The read-only database key on the free vector-database tier could still write, which I accepted as a stated limitation for the demo.

Contents

Declaration 2

Acknowledgements 3

Executive Summary 4

Contents 6

List of Figures 10

List of Tables 11

Abbreviations 13

1. Introduction 14

    1.1 The Indian OTT Market Has a Choice Problem 14

    1.2 The Business Problem 14

    1.3 What I Set Out to Build 15

    1.4 Research Questions 15

    1.5 Scope and Constraints 16

    1.6 How This Report Is Organized 16

2. Background and Literature 18

    2.1 Two Ways to Recommend 18

    2.2 The Two-Stage Design 18

    2.3 Text Embeddings for Content 19

    2.4 Neural Collaborative Filtering, and a Word of Caution 19

    2.5 Learning to Rank 19

    2.6 Context-Aware Recommendation 19

    2.7 Bandits for New Items 20

    2.8 Vector Databases 20

    2.9 Measuring Recommenders 20

    2.10 Where This Project Fits 21

3. Data 22

3.1 Why MovieLens....22
3.2 What Counts as a "Like"....23
3.3 Adding Movie Descriptions from TMDB....23
3.4 What the Data Cannot Tell Us....23
3.5 Why I Did Not Split the Data at Random....24
3.6 Locking the Test Set....25
4. Methodology....27
4.1 The Big Picture: Two Stages, Then a Shelf for New Titles....27
4.2 How I Measured Success....28
4.3 Stage One: Finding Candidates....29
4.4 Stage Two: Ranking the Candidates....31
4.5 New Releases and the Bandit....33
4.6 Putting It Online....34
4.7 Tools....35
5. Results....36
5.1 The Five Headline Results....36
5.2 Candidate Generation....37
5.3 Ranking....38
5.4 Does Context Help?....39
5.5 New Releases and the Bandit....40
5.6 Serving in Practice....41
5.7 Where I Was Wrong....43
6. What This Means for an OTT Business....45
6.1 The Ranker Is Where the Value Is....45
6.2 Do Not Pay for Context Before You Can Measure It....45
6.3 Promoting New Releases Has a Price, and It Is Worth Paying....46
6.4 What It Costs to Serve....46

6.5 What a Real Launch Would Measure 48
6.6 Build or Buy? 48
6.7 My Recommendations 48
7. Limitations, Risks and Ethics 50
7.1 The Data Is a Stand-In 50
7.2 The Evaluation Has Limits 50
7.3 Engineering Shortcuts and Their Risks 51
7.4 A Lesson in Cloud Governance 51
7.5 Ethics and Responsible Recommendation 51
7.6 Use of an AI Assistant 52
8. Conclusion and Future Work 53
8.1 Answers to the Research Questions 53
8.2 What I Take Away 53
8.3 Future Work 54
8.4 Closing 54
References 56
Appendix A. Reproducibility 60
A.1 Environment 60
A.2 Data 60
A.3 Scripts by Module 60
A.4 The One-Shot Test Run 61
A.5 Serving Measurements 61
Appendix B. Full Result Tables 62
Appendix C. Predictions Made Before the Results 65
Appendix D. Security and Access Design 67
D.1 Who Can Do What 67
D.2 How Secrets Were Handled 67

D.3 Safe Teardown 68
Appendix E. Deployment Runbook (Summary) 69
Appendix F. The Demo 70
Appendix G. Glossary for Non-Technical Readers 71
Appendix H. Tools Used and AI Assistance 73
H.1 Software and Services 73
H.2 Use of an AI Assistant 73

List of Figures

Figure 3.1 *The two ways of splitting the data: one calendar cutoff for new-release questions, and a per-user time split for everything else* 25

Figure 4.1 *System architecture. Models are trained offline on a laptop with TensorFlow; the online part on AWS runs with numpy only* 27

Figure 4.2 *From the whole catalog to the 20 titles a user sees* 28

Figure 5.1 *The five headline results with their 95% intervals. Left: test set. Right: bandit simulation, not an A/B test* 37

Figure 5.2 *How much of each user's future likes lands in the top 200, test set. Orange bars count only likes on movies that had no likes in training* 38

Figure 5.3 *End-to-end NDCG@10 on the test set when the same 230 candidates are ordered by different methods (95% intervals)* 39

Figure 5.4 *Simulated likes per 1,000 impressions on the 761 titles with at least five ratings, test window (SIMULATION, NOT AN A/B TEST)* 41

Figure 5.5 *Where the server time goes: median per stage, 200 warm requests per mode* 42

Figure 5.6 *Live search quality by Zilliz search level, 100 users* 43

Figure F.1 *Section B of the demo page, redrawn from the saved page: the same user's top 18 under a SIMULATED tv and a SIMULATED mobile. The synthetic rule removed drama-without-comedy likes on mobile during training* 70

List of Tables

Table 1.1 *Required components and how they were delivered* 15

Table 3.1 *The 20,000-user sample at a glance* 22

Table 3.2 *The per-user time split* 25

Table 4.1 *The 37 inputs of the ranker (plus 4 SIMULATED device columns in the M3 variants)* 31

Table 5.1 *Headline results on the test set, with the predictions committed beforehand* 36

Table 5.2 *The context ladder on the test set (end-to-end NDCG@10; secondary except P3)* 39

Table 5.3 *Measured latency, 200 warm requests per mode (server time is measured inside the function)* 42

Table 6.1 *Estimated serving cost per million home-screen requests* 46

Table 6.2 *A proposed A/B test plan* 48

Table A.1 *Where each part of the work lives* 60

Table B.1 *Ordering the 230 candidates, test set, 19,530 users (mean, 95% interval in brackets)* 62

Table B.2 *Retrieval over the whole catalog, test set, 19,530 users* 62

Table B.3 *Bandit simulation on the test window: simulated likes per 1,000 impressions (SIMULATION, NOT AN A/B TEST)* 63

Table B.4 *Cost of reserving slots for new releases, M2 ranking, test set (difference against no reservation)* 63

Table B.5 *Latency of the deployed service, 200 warm requests per mode (milliseconds)* 64

Table B.6 *Live search quality by Zilliz search level, 100 users* 64

Table B.7 *The context ladder on validation, 15,530 held-out users, out-of-fold scores (supporting evidence)* 64

Table C.1 *Pre-registration files and the commits that fixed them* 65

Table C.2 *The headline test claims: prediction against result* 65

Table C.3 *Serving latency: prediction against result* 66

Table D.1 *The identities and their permissions* 67

Table E.1 *Deployment steps* 69

Table G.1 *Terms used in this report* 71

Abbreviations

Abbreviation	Meaning
A/B test	A controlled online experiment comparing two versions with randomly assigned users
ANN	Approximate nearest neighbor (search)
API	Application programming interface
AWS	Amazon Web Services
CTV	Connected TV
DPDP Act	Digital Personal Data Protection Act, 2023 (India)
IAM	Identity and Access Management (AWS permissions)
kNN	k-nearest neighbors
MiB	Mebibyte (1,048,576 bytes)
ms	Millisecond
NCF	Neural collaborative filtering
NDCG	Normalized discounted cumulative gain
OTT	Over-the-top (streaming delivered over the internet)
RQ	Research question
SBERT	Sentence-BERT
SCP	Service control policy (AWS Organizations)
SSM	AWS Systems Manager (Parameter Store)
TF	TensorFlow
TMDB	The Movie Database
UCB	Upper confidence bound
UTC	Coordinated Universal Time

1. Introduction

1.1 The Indian OTT Market Has a Choice Problem

India is now one of the largest streaming audiences in the world. According to the Ormax OTT Audience Report 2025, 601.2 million Indians watch content on OTT platforms, about 41% of the population. Active paid subscriptions stand at 148.2 million (a figure that includes telecom bundles and aggregators), and connected TV users grew 87% in a single year to 129.2 million (afaqs!, 2025).

As a result, platforms are competing hard for the same viewer's evening. Most households have access to more than one service, and each service offers thousands of movies and series in several languages.

That abundance creates a problem. A viewer opens the app, sees a home screen with a couple of dozen titles and decides within seconds whether anything looks worth watching. If nothing does, they scroll for a while and then switch to another app, or to YouTube, or close the app altogether.

Recommendation systems exist to solve exactly this. Netflix has published one of the few public estimates of what they are worth: about 80% of the hours streamed on Netflix came from recommendations, and the company estimated that personalization and recommendations together save it more than a billion US dollars a year, mostly through lower cancellations (Gomez-Uribe & Hunt, 2015). Even if an Indian platform gets a fraction of that effect, the business case is clear.

1.2 The Business Problem

From a business point of view, an OTT recommender has to do four things well.

1. Put the right titles on the first screen. The first row matters most, because many users never scroll past it. The system has to pick a handful of titles out of thousands, for each user, every time they open the app.

2. Give new releases a fair chance. Platforms spend heavily on new content and need people to see it. However, a brand-new title has no viewing history, so a system that learns from history tends to ignore it. This is usually called the cold-start problem.

3. Use the moment, not just the person. What someone wants at 11 pm on a phone may differ from what they want on Sunday afternoon on a TV. In principle, context such as time

and device should improve recommendations. Whether it does in practice is an open question that this project tests.

4. Run fast and cheaply at scale. A recommendation that arrives after the home screen has loaded is useless. It also has to be cheap enough to compute for millions of users several times a day.

1.3 What I Set Out to Build

The project brief asked for a hyper-personalized OTT recommendation engine with four specific components. Table 1.1 maps each one to what I built and where it is covered in this report.

Table 1.1 *Required components and how they were delivered*

Required component	What I built	Chapters
Candidate generation with a two-tower model and Sentence-BERT embeddings in Milvus	ID-only two-tower model in TensorFlow for the main candidates, plus a 30-title new-release lane from Sentence-BERT descriptions; both sets of vectors loaded into Zilliz Cloud (hosted Milvus)	4.3, 5.2, 5.6
Ranking with NCF and an XGBoost ranker using context (device, hour, session length)	NCF in TensorFlow used as an input to an XGBoost ranker with 37 inputs; a ladder of models that adds hour, day and session; device tested with SIMULATED values	4.4, 5.3, 5.4
A UCB multi-armed bandit for new releases	UCB with a tuned constant filling 2 of 20 slots; compared with five other strategies in a simulation	4.5, 5.5
Serving on AWS Lambda and DynamoDB	A numpy-only Lambda function in Frankfurt with DynamoDB, plus live vector search in Zilliz; deployed, measured and torn down	4.6, 5.6

The brief also set some ground rules that shaped the work: TensorFlow for the neural models, data split by time and never at random, every model compared with simple baselines, and simulated parts clearly labeled.

1.4 Research Questions

I turned the business problem into six questions. RQ1 to RQ4 match the five headline claims (P1 to P5) that were fixed, with predicted answers, before the final test (Section 3.6). RQ5 is answered from the supporting evidence, and RQ6 from the deployed system.

- **RQ1.** Does a learned ranker put better titles at the top than the candidate generator's own order? (P1, P2)
- **RQ2.** Does context (time of day, day of week, session) improve the ranking? (P3)
- **RQ3.** Does the two-tower model find more of a user's future likes than a classic item-based method? (P4)
- **RQ4.** Which bandit strategy gives new releases the most successful exposure, and does tuning matter? (P5, simulation)
- **RQ5.** Can a content lane surface new titles that the other methods cannot see at all?
- **RQ6.** Can the whole system run on low-cost serverless infrastructure with acceptable speed?

Chapter 5 answers each of them with numbers, and Chapter 8 sums up the answers.

1.5 Scope and Constraints

This was a one-month MBA project, built by one person on a laptop. That set some hard limits, and I think it is fair to state them up front.

- **Hardware.** A laptop with 16 GB of RAM (about 9 GB usable under Windows Subsystem for Linux) and no GPU. Everything was sized to fit: a 20,000-user sample instead of the full dataset, and 64-number vectors instead of the larger ones used in industry.
- **Budget.** Only free tiers and free credits: AWS free tier, the free Zilliz Cloud cluster and the free TMDB API. The cloud deployment was sized to stay within these allowances; Chapter 6 estimates what it would cost beyond them.
- **Data.** No real OTT viewing data is public, so I used MovieLens ratings with TMDB descriptions (Chapter 3). There is no device in this data, so every device value is SIMULATED.
- **Tools.** TensorFlow was required by my guide for the neural models. The deployed function had to work without it, which shaped the serving design (Section 4.6).

Within these limits, my aim was not to beat the state of the art. It was to build the whole pipeline end to end, measure every part honestly, and be clear about what the numbers do and do not show.

1.6 How This Report Is Organized

Chapter 2 reviews the methods this project builds on. Chapter 3 describes the data and how it was split. Chapter 4 explains how each component was built and how it was evaluated.

Chapter 5 presents the results, starting with the five headline findings. Chapter 6 discusses what the results mean for an OTT business, including costs. Chapter 7 covers the limitations, risks and ethical questions, and Chapter 8 concludes with the answers to the research questions and ideas for future work. The appendices hold the full tables, the pre-registered predictions, the security design, the demo, a glossary and a statement on the tools I used.

2. Background and Literature

This chapter covers the ideas the project builds on. I kept it to what is needed to follow the later chapters, and for each idea I note where it shows up in my own design.

2.1 Two Ways to Recommend

There are two classic families of recommenders.

Collaborative filtering learns from behavior: people who liked the same things in the past will probably like the same things in the future. It needs no knowledge of what a movie is about. Item-based nearest neighbors, which recommends movies similar to the ones a user already liked (similar meaning "liked by the same people"), was one of the first methods to work at web scale (Sarwar et al., 2001). Matrix factorization, popularized during the Netflix Prize, instead learns a short vector for every user and every movie so that their product predicts a rating (Koren et al., 2009). Most modern models, including the two-tower model in this project, are descendants of that idea.

Content-based filtering looks at the items themselves: genres, cast, descriptions. It can recommend a movie on the day it is added to the catalog, which collaborative filtering cannot. However, a description says less about taste than the behavior of thousands of other viewers, so content methods are usually weaker for established titles.

Real systems combine both. In my system, the main candidates come from a collaborative model and the new-release lane comes from a content model (Section 4.3).

2.2 The Two-Stage Design

Scoring an entire catalog with a heavy model for every request does not scale. YouTube's published design splits the job in two: a fast candidate generator narrows millions of videos to hundreds, and a ranker with many more features orders those hundreds (Covington et al., 2016). This two-stage pattern is now standard, and it is the backbone of my project (Figure 4.1).

The candidate generator is often a **two-tower model**: one neural network encodes the user, another encodes the item, and the score is how closely the two vectors align. Because item vectors can be computed in advance, finding the best candidates becomes a nearest-neighbor search. One known weakness is that training with in-batch negatives over-penalizes popular items, since they appear in many batches. Yi et al. (2019) proposed a correction based on

each item's sampling frequency, which I used (and which turned out to be essential, Section 4.3).

2.3 Text Embeddings for Content

To compare movie descriptions, they first have to become numbers. Sentence-BERT fine-tunes a BERT language model so that sentences with similar meaning end up as nearby vectors, which makes comparing two texts a simple dot product (Reimers & Gurevych, 2019). I used a small, fast version of it to embed TMDB plot descriptions for the new-release lane.

2.4 Neural Collaborative Filtering, and a Word of Caution

Neural collaborative filtering (NCF) replaces the plain dot product of matrix factorization with a neural network that can learn more complex patterns between user and item vectors (He et al., 2017). It became a common baseline in research and is one of the components the project brief asked for.

However, the field has a reproducibility problem. When Ferrari Dacrema et al. (2019) re-ran a set of recent neural recommendation papers, most of the reproducible ones could be beaten by simple, well-tuned older methods such as item-based nearest neighbors. This finding shaped how I worked. Every model in this project had to beat simple baselines on the same data, with intervals, before I believed it. As it turned out, NCF on its own did not beat the two-tower model's order, so I used it as one input to the ranker instead (Section 4.4).

2.5 Learning to Rank

Ranking is not the same as predicting a rating. What matters is the order of the list, and mistakes near the top matter more than mistakes at position 150. Learning-to-rank methods optimize the order directly. The LambdaMART family trains gradient-boosted decision trees on pairs of items for the same user, weighting each pair by how much swapping them would change a ranking metric such as NDCG (Burges, 2010). XGBoost is a fast, widely used implementation of gradient boosting that supports this kind of ranking objective (Chen & Guestrin, 2016). My ranker is an XGBoost model of this type.

2.6 Context-Aware Recommendation

Context-aware recommenders add information about the situation, such as time, location, device or company, to information about the user and item (Adomavicius & Tuzhilin, 2011). The idea is intuitive: the same person may want different things on a weekday morning

commute and on a Saturday night. The practical question is how much context adds once a model already knows the user's taste and recent activity. That is what the context ladder in this project measures (Section 4.4).

2.7 Bandits for New Items

A multi-armed bandit is a model of repeated choice under uncertainty. Each option (an "arm") has an unknown reward rate, and the decision maker has to balance trying uncertain options with using the ones that already look good. Lattimore and Szepesvári (2020) give a full treatment of the theory.

Two strategies matter here. **UCB** (Upper Confidence Bound) adds an optimism bonus to each arm's observed average, which shrinks as the arm is tried more often; Auer et al. (2002) proved that a simple version, UCB1, performs close to the best possible in the long run. **Thompson sampling**, one of the oldest ideas in the field (Thompson, 1933), picks each arm with the probability that it is the best one, given what has been seen so far.

Evaluating a bandit offline is hard. You only see the outcome of what was actually shown. Li et al. (2010) showed how to replay logged data fairly, but only when the logged choices were made at random. MovieLens ratings are not random exposures, which is why my bandit results come from a simulation (Section 4.5).

2.8 Vector Databases

A two-tower model turns retrieval into a search for the item vectors closest to a user vector. Doing that exactly means comparing against every item, which gets slow as catalogs grow. Approximate nearest-neighbor indexes trade a little accuracy for a lot of speed. One of the most widely used is HNSW, a layered graph that lets the search jump quickly toward the right neighborhood (Malkov & Yashunin, 2020). Milvus is an open-source database built around such indexes (Wang et al., 2021), and Zilliz Cloud, which I used, is a hosted version of it. The price of approximation shows up in my results as live search returning about 92% of the exact candidates (Section 5.6).

2.9 Measuring Recommenders

Offline metrics such as Recall@K and NDCG (Järvelin & Kekäläinen, 2002) are the standard way to compare recommenders on historical data. They are convenient, but they measure agreement with past behavior, not business value. Jannach and Jugovac (2019) reviewed how companies actually measure the value of recommenders (click-through, sales, engagement,

retention) and found that offline accuracy gains do not always translate into business gains. The reliable way to measure business impact is a controlled online experiment, an A/B test, run with care about sample size, duration and guardrail metrics (Kohavi et al., 2020).

This gap between offline and online evaluation runs through my whole report. Every result in Chapter 5 is offline. Chapter 6 discusses what an online test would need to confirm.

2.10 Where This Project Fits

Most published work studies one component at a time: a better retriever, a better ranker or a better bandit, usually measured offline on a single metric. In contrast, there are few end-to-end, openly documented examples that take a recommender from raw data all the way to a deployed, measured service, with every claim checked against baselines and a locked test set.

My project tries to fill that gap at a small scale. It builds all four components the brief required, connects them into one working service, and evaluates each part with the same discipline: predictions written down first, simple baselines, intervals, and one look at the test set. Its value is less in any single model and more in showing how the pieces fit together, what each one actually contributes, and what it costs to run.

3. Data

3.1 Why MovieLens

A real OTT platform logs what people watch, for how long, on which device and at what time. None of that data is public. Netflix, Hotstar or Prime Video do not share it, and for good reason.

So I needed a public stand-in. The best known one is MovieLens, a set of movie ratings collected by the GroupLens research lab at the University of Minnesota (Harper & Konstan, 2015). I used the MovieLens 25M release. It holds 25 million ratings from about 162,000 users on about 62,000 movies, given between 1995 and 2019.

MovieLens is not a streaming log. It records ratings on a 0.5 to 5 star scale, not plays. However, it is the closest thing to viewing data that is free, large and widely used in research. That last point matters: results on MovieLens can be compared with published work.

To keep everything runnable on a laptop with 16 GB of RAM (about 9 GB usable under WSL), I worked with a random sample of 20,000 users. The sample was drawn once with a fixed seed (42), and every rating of each sampled user was kept. Table 3.1 sums it up.

Table 3.1 *The 20,000-user sample at a glance*

Item	Value
Users	20,000 (every MovieLens user has at least 20 ratings)
Ratings	3,101,914
Movies rated	33,392
Period	January 1996 to November 2019
Empty cells in the user by movie grid (sparsity)	99.5%
Ratings per user	median 72, mean 155, maximum 8,913
Ratings per movie	median 4; 9,997 movies have exactly one rating
Share of ratings that count as a "like" (4 stars or more)	49.9%

Two things in this table shape the rest of the project.

First, the data is extremely sparse. A typical user has rated 72 of the 33,392 movies, which is about 0.2%. Any model has to guess the other 99.8%.

Second, the catalog has a very long tail. Half of the movies have four ratings or fewer. Popular titles collect thousands. A recommender that simply pushes popular titles will look decent on paper and still be useless for most of the catalog (more on this in Chapter 5).

3.2 What Counts as a "Like"

The models need a yes or no signal. I treated a rating of 4 stars or more as a like and anything lower as a non-like. This splits the data almost exactly in half (49.9% likes).

The choice has a catch. A 2-star rating still means the person watched the movie. Thus, a "non-like" is not the same as "not interested". I kept this in mind throughout: every result in this report is about predicting what a user will like, not what they will click or watch.

3.3 Adding Movie Descriptions from TMDB

Ratings alone tell you nothing about a movie that nobody has rated yet. That is exactly the situation of a new release on an OTT platform.

To handle new titles, I added plot descriptions and release dates from The Movie Database (TMDB). MovieLens ships a link file that maps each movie to its TMDB id, so I fetched the details for all 33,319 unique ids through the TMDB API. The fetch took 36 minutes; 32,859 requests succeeded and 440 ids no longer existed on TMDB.

In the end, 98.4% of the movies have a usable description of at least 40 characters, and 32,902 have a release date. For the remaining 519 movies I fell back to the title, the genres and, where available, MovieLens tags. These descriptions later become text embeddings, which let the system match a brand-new movie to a user's taste without a single rating (Chapter 4).

This product uses the TMDB API but is not endorsed or certified by TMDB.

3.4 What the Data Cannot Tell Us

Before building anything, I went through the data looking for traps. I found four that matter.

1. Ratings are not viewing. MovieLens users rate movies on a website. Many of them rate dozens of old favorites in one sitting. In the sample, 58.2% of all ratings sit inside bursts of 30 or more ratings within 10 minutes, and 85% fall inside "sessions" of 20 or more ratings. A real viewer does not watch 30 movies in 10 minutes. So when I talk about sessions later, they are rating sprees, not viewing sessions.

2. There is no device. The project brief asks for device as a context feature. MovieLens does not record one. Therefore every device value in this report is SIMULATED, and I label it that way each time it appears.

3. Time is in UTC. Timestamps carry no time zone, so "hour of day" is UTC hour, not the user's local evening. The busiest UTC hours are 19:00 to 21:00 (about 6% of ratings each), which fits an American evening, but I cannot confirm where users live.

4. The users are enthusiasts. People who rate 72 movies on a research website are not a random slice of Indian OTT subscribers. The patterns here are a reasonable proxy for taste, not a measurement of any real platform's audience.

None of these are fatal. However, they set the limits of what I can claim, and Chapter 7 comes back to them.

3.5 Why I Did Not Split the Data at Random

To test a recommender honestly, you train it on the past and test it on the future. Splitting ratings at random would let the model peek at a user's later behavior while learning, which inflates every score.

My first attempt was a single calendar cutoff for everyone: train on ratings up to 3 February 2015, validate up to 28 March 2017, and test on the rest. It failed in an interesting way. Most users in the test period had never appeared in the training period at all: 2,610 of the 2,986 test users were new, and 90% of test ratings came from them. The reason is that many MovieLens users rate their whole history in one go and then leave. A calendar cutoff puts each of them entirely on one side of the line.

That makes a global cutoff useless for measuring personalization, because you cannot personalize for someone you have never seen. Interestingly, the same property makes it perfect for one specific question: how well can the system handle movies released after the cutoff? So I kept the calendar split for the new-release work (Chapters 4 and 5) and built a second split for everything else.

The second split works per user. I sorted each user's ratings by time, kept the last 20% for testing, the 10% before that for validation, and the oldest 70% for training. Every user now appears in all three parts, and the model is always asked to predict a user's future from their own past. Table 3.2 shows the result, and Figure 3.1 shows both splits on a timeline.

Table 3.2 *The per-user time split*

Part	Ratings	Used for
Training (oldest 70% of each user)	2,187,571	fitting every model
Validation (next 10%)	301,690	tuning, model choice, and training the ranker; 18,530 users have at least one like here
Test (newest 20%)	612,653	one final measurement; 19,530 users have at least one like here

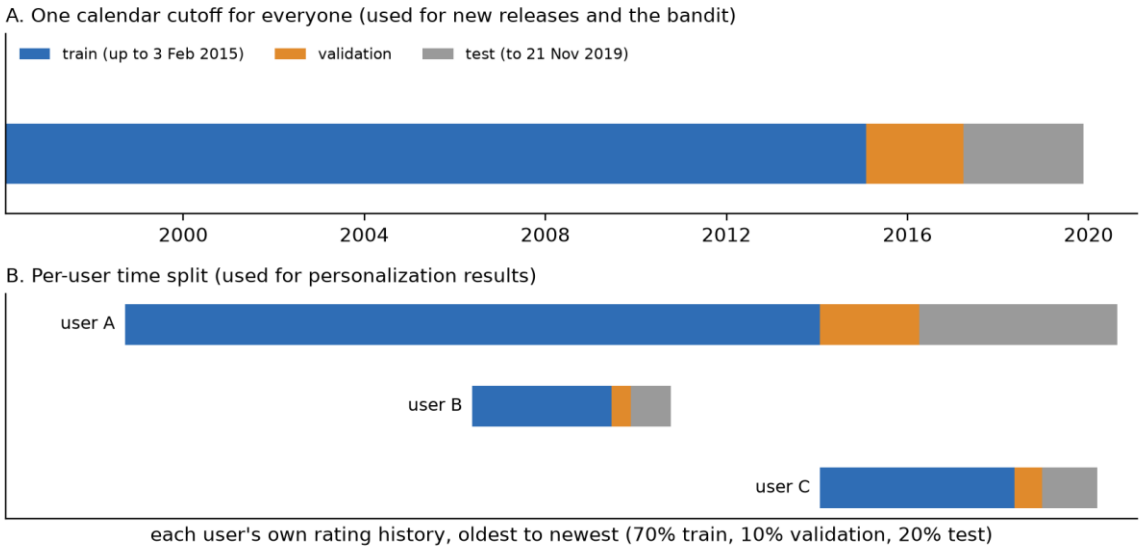


Figure 3.1 *The two ways of splitting the data: one calendar cutoff for new-release questions, and a per-user time split for everything else*

One side effect is worth stating. Under the per-user split, almost every movie in the validation and test periods already existed in training (only 0.36% of validation likes were on movies unseen in training). A per-user split does not create new releases. That is why the new-release lane and the bandit are judged on the calendar split instead.

3.6 Locking the Test Set

The easiest way to fool yourself in machine learning is to look at the test data while making decisions. Every peek nudges the choices toward whatever happens to work on that particular test set.

To make peeking impossible rather than just discouraged, I locked the test portion in code. Any attempt to read it raised an error, automated checks scanned all tuning code for

references to it, and every script was also run once with test reads made fatal. All tuning, model selection and the ranker's training used only the training and validation parts.

The lock was opened exactly once, on 26 October 2026, by a single script with my written predictions committed beforehand. That run produced the headline results in Chapter 5. It can never run again: the script refuses to start if its own log already exists in the project history.

To be fair, this level of care is unusual for a student project. Still, I wanted the main numbers in this report to be ones I could defend in front of anyone.

4. Methodology

4.1 The Big Picture: Two Stages, Then a Shelf for New Titles

An OTT home screen shows maybe 20 titles at a time, picked from a catalog of thousands. Scoring every title for every user with a heavy model on every visit is slow and expensive. So most large recommenders split the work in two (Covington et al., 2016).

The first stage, candidate generation, is quick and rough. It narrows 33,392 movies down to a few hundred that might interest this user. The second stage, ranking, is slower and smarter. It looks closely at those few hundred, using many more signals, and puts them in order.

My system follows the same idea, with one addition the project brief asked for: a small reserved shelf for new releases, filled by a multi-armed bandit. Figure 4.1 shows the whole system, and Figure 4.2 shows how the numbers shrink from catalog to screen.

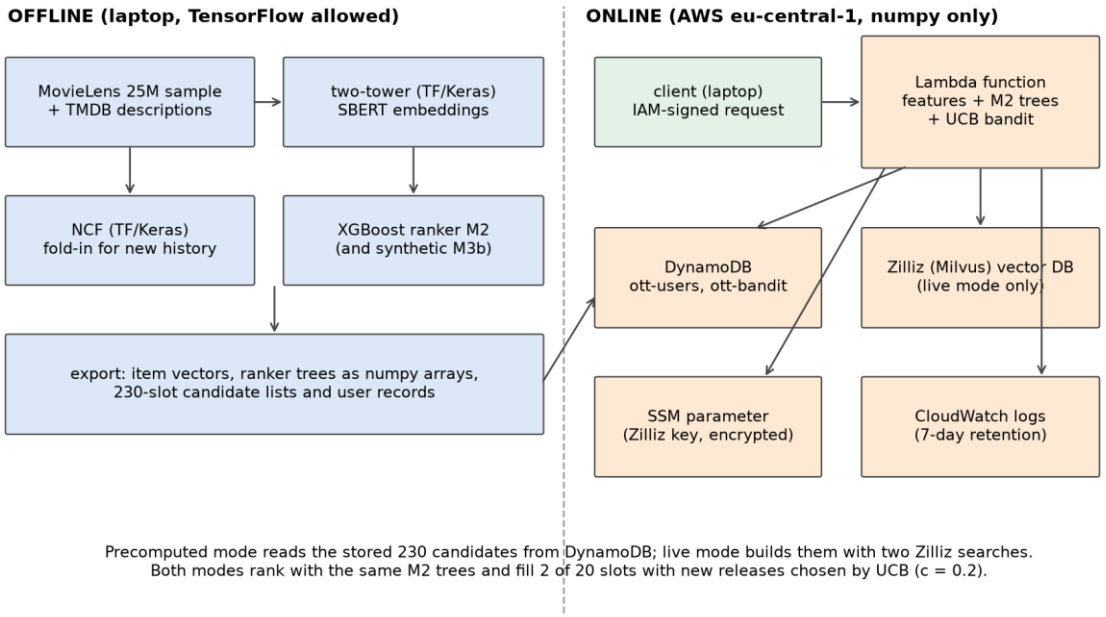


Figure 4.1 *System architecture. Models are trained offline on a laptop with TensorFlow; the online part on AWS runs with numpy only*

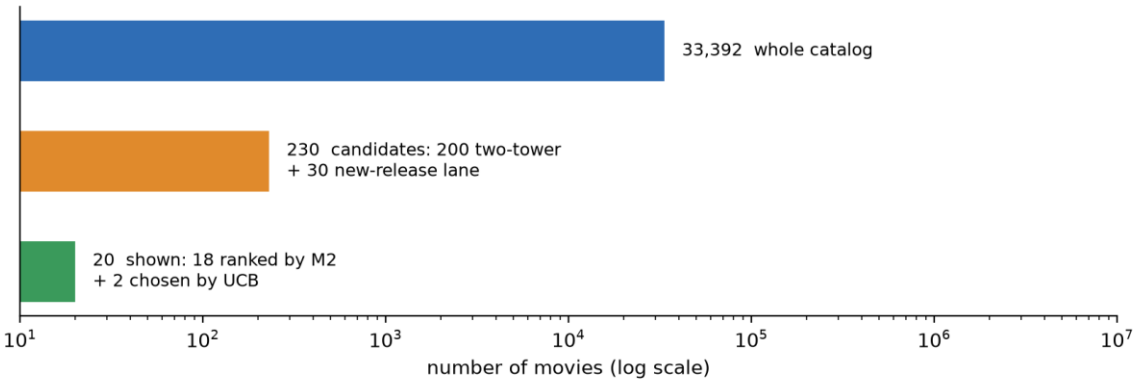


Figure 4.2 *From the whole catalog to the 20 titles a user sees*

Everything heavy happens offline on the laptop: training the models, computing embeddings and preparing candidate lists. The online part on AWS Lambda only loads precomputed numbers and does light arithmetic. This split also follows the project rule that TensorFlow runs offline only.

4.2 How I Measured Success

Before building models, I fixed how they would be judged. Otherwise it is too easy to pick whatever metric makes the result look good.

The metrics. I used two standard ones from information retrieval.

- **Recall@K** asks: of all the movies this user went on to like, what share appeared in their top K? Recall@200 judges candidate generation (did the right movies make it into the pool at all?), and Recall@20 judges what ends up on the screen.
- **NDCG@10** (Normalized Discounted Cumulative Gain) looks only at the top 10 and gives more credit for a liked movie at position 1 than at position 10 (Järvelin & Kekäläinen, 2002). It is the closest single number to "is the first row of the home screen any good?"

Both are averaged over users. I also report coverage (how many different movies a method recommends across all users), because a method that shows everyone the same blockbusters can score decently while being boring.

End-to-end, not just "reachable". A ranker can only reorder what candidate generation gives it. If a liked movie never made it into the 230 candidates, no ranker can find it. I report end-to-end numbers, where every liked movie counts, as the main view. A separate "reachable" view counts only likes that were inside the candidate list.

Baselines. Every model had to beat simple alternatives: random order, popularity (most-liked movies first) and item-based nearest neighbors, an older but strong method that recommends movies similar to the ones a user liked (Sarwar et al., 2001). A model that cannot beat these is not worth its complexity.

Uncertainty. Each difference between two methods is reported with a 95% interval from a paired bootstrap over users (Efron & Tibshirani, 1993). Simply put, I resampled the users many times and checked how much the difference moved. If the interval excludes zero, the difference is unlikely to be luck.

Predictions before results. For every module from candidate generation onward, I wrote down what I expected (direction, size and confidence) and committed it to the project history before running anything. This keeps me honest. A result I predicted carries more weight than one I explained after the fact, and the misses are part of the record (Chapter 5 lists them).

One look at the test set. As described in Section 3.6, the test portion was locked during all development. The final evaluation protocol was written and committed before the lock was opened. It fixed the five headline claims (P1 to P5 in Chapter 5), the exact comparisons and the rule for reading them. Only those five are findings. Everything else in the test run is secondary.

4.3 Stage One: Finding Candidates

A two-tower model in TensorFlow

The main source of candidates is a two-tower model, built in TensorFlow and Keras as the project required. One tower turns each movie into a vector of 64 numbers. The other tower turns each user into a vector of the same size. A movie whose vector points in the same direction as the user's vector is a good candidate, and finding those is fast even for a large catalog.

The user tower does not need a separate ID for each user. It takes the user's last 20 liked movies, averages their vectors and passes the average through one dense layer. This has a practical benefit: when a user likes a new movie, their vector can be updated instantly with a little numpy arithmetic, without retraining anything.

Training used pairs of (user history, next liked movie), 1.09 million of them, with an in-batch softmax loss: for each user in a batch of 2,048, the model learns to score the movie they actually liked above the movies the other users liked. Because popular movies show up in

many batches, they get pushed down unfairly. I corrected for this with the log-popularity adjustment from Yi et al. (2019), usually called logQ correction. It turned out to be essential: switching it off took validation Recall@200 of the ID-plus-description model from 0.635 down to 0.263, worse than plain popularity.

I tried three versions: movie ID only, movie description only, and ID plus description. On validation, the ID-only model was best for movies that already had ratings (Recall@200 0.650 against 0.584 for item-kNN). Adding the description did not help those movies and hurt slightly (by 0.015). So the main lane uses the ID-only model.

A content lane for new movies

An ID-only model has a blind spot: a movie nobody has rated has no learned vector. It can never be recommended. On an OTT platform, that blind spot covers exactly the titles the business most wants to promote, the new releases.

To fill the gap, I turned each movie's TMDB description into a 384-number embedding with Sentence-BERT, a model that maps sentences with similar meaning to nearby vectors (Reimers & Gurevych, 2019). I used the small all-MiniLM-L6-v2 version, which runs on a laptop CPU. A user's taste profile is then the average embedding of the movies they liked, and new movies are scored by how close their description sits to that profile.

One detail made a big difference. Raw sentence embeddings share a large common direction, so every user's average profile drifts toward "generic movie". Subtracting the average embedding of all movies (centering) fixed this and multiplied validation NDCG@10 by about five. All content scores in this report use centered embeddings.

The final candidate list

Each user gets 230 candidates:

- **200 from the two-tower model** (movies the user has not rated yet), and
- **30 from the content lane**, searched only among movies with no likes in the training data, and appended after the 200 rather than replacing any of them.

I tested several other mixes. Giving the content model a share of the 200 main slots cost accuracy on established movies, and almost all of that cost came from losing slots, not from content being bad. Appending the lane kept the main list intact while adding the new-release coverage. Finally, a candidate released after the moment of the request is removed, so the system never "recommends" a movie that did not exist yet.

4.4 Stage Two: Ranking the Candidates

NCF as an extra opinion

Neural Collaborative Filtering, or NCF (He et al., 2017), learns a vector for every user and every movie from the full rating history and predicts a like with a small neural network. I built the NeuMF variant in TensorFlow (two 32-number embeddings per user and movie, and three hidden layers of 64, 32 and 16 units), trained with four sampled negative movies for each liked one.

On its own, NCF ordered the candidates worse than the two-tower model (validation NDCG@10 0.089 against 0.123). However, it looks at a different thing: long-term taste, while the two-tower model looks at recent likes. So instead of using NCF as a recommender, I used its score as one input to the ranker.

For the final test, NCF's user vectors were refreshed with each user's validation-period likes without retraining the rest of the network (a "fold-in"). Item vectors and the network itself stayed frozen.

The XGBoost ranker

The ranker is a gradient-boosted tree model from the XGBoost library (Chen & Guestrin, 2016), trained with a ranking objective in the LambdaMART family (Burges, 2010). In plain words, it learns from pairs of candidates for the same user, one liked and one not, and adjusts itself to put the liked one higher, with extra weight on mistakes near the top of the list.

Table 4.1 lists its inputs.

Table 4.1 *The 37 inputs of the ranker (plus 4 SIMULATED device columns in the M3 variants)*

Group	Inputs
Two-tower	similarity score; position in the two-tower list
NCF	predicted like (logit); whether NCF knows this user; whether it knows this movie
Popularity	number of likes the movie had before the request time
Content	similarity between the movie's description and the user's profile; match between the movie's genres and the user's favorite genres
Movie	age in days at the request time; whether it came from the new-release lane; 19 genre flags
Time context	hour of day and day of week (UTC)

Group	Inputs
Session context	minutes since the last activity; whether the user is mid-session; session length so far; share of likes in the session; genre and content match with the session

Every input is computed only from data before the request time. Automated tests check this by adding future ratings and confirming that no input changes.

The ranker was trained on the validation period. Each validation user's 230 candidates became rows, labeled 1 if the user liked the movie in the validation period. That gave 3.93 million rows with 1.82% positives. The two-tower and NCF scores in these rows came from models that never saw the validation period, so the ranker learned from honest, out-of-sample scores. For the validation results, I trained on four fifths of the users and scored the other fifth, five times over. For the test, each ranker was retrained once on all 18,530 validation users (M0 with 188 trees, M1 with 65 and M2 with 351).

Does context help? The ablation ladder

The brief asks for context features: device, hour and session length. To measure what each one adds, I trained a ladder of rankers that differ only in their inputs:

- **M0:** no context;
- **M1:** M0 plus hour of day and day of week;
- **M2:** M1 plus the session features (the main model);
- **M3a:** M2 plus a SIMULATED device that is random and unrelated to behavior. This is a check: it should add nothing.

Comparing neighbors on the ladder shows what each group of features is worth.

Device needs special care. MovieLens has no device, so any "device effect" I show would be invented. I still wanted to prove the pipeline could learn a device effect if one existed. So I built a clearly labeled synthetic scenario: on SIMULATED mobile requests, 60% of the likes of drama movies without comedy were removed from the training labels. A model with device as an input (**M3b**) should pick this up, and the same model with the device values shuffled between users should not. **This is a synthetic device effect, a pipeline demonstration, not a finding.**

4.5 New Releases and the Bandit

The problem

A brand-new title has no ratings, so the ranker has little to go on and tends to bury it. In my validation runs, new-release candidates never reached the ranker's top 20. Yet a platform pays for new content and needs people to see it.

The standard answer is to reserve a couple of slots for new titles and let a multi-armed bandit choose what goes there. A bandit treats each new title as a slot machine with an unknown payout (the like rate). It has to balance exploring titles it knows little about with exploiting titles that already look good.

I used UCB, short for Upper Confidence Bound (Auer et al., 2002). For each title it adds a bonus to the observed like rate: the fewer times a title has been shown, the bigger the bonus. Titles that are either good or still uncertain get shown. The size of the bonus is set by a constant, c. I compared UCB with the textbook constant (UCB1), a tuned constant, UCB1-Tuned, Thompson sampling (Thompson, 1933), epsilon-greedy, plain greedy and random.

Why a simulation

Testing a bandit properly needs live users: you show something and see what they do. Offline "replay" methods exist, but they need logs where the shown item was chosen at random (Li et al., 2010). MovieLens ratings are people's own choices, not random exposures. So I evaluated the bandit with a simulation, and every bandit number in this report carries the label **SIMULATION, NOT AN A/B TEST**.

The simulation was built from real data where possible:

- **Arms:** movies released (by TMDB date) shortly before or during the evaluation window and never rated before it.
- **Hidden like rates:** each movie's real ratings in the window, pulled toward the overall like rate (equivalent to five extra average ratings) so that thinly rated movies do not look extreme. A fresh draw is made for each of 30 repeated runs.
- **Requests:** the real session starts in the window (a new session begins after 30 minutes of silence).
- **Rules:** 2 reserved slots per request; a movie stays on the new-release shelf for 365 days after release; a "like" is drawn from the hidden rate. The reward means "liked, given that the user rated it" and ignores whether they would have clicked at all.

The bandit was judged on the calendar split from Section 3.5, because that is where real new releases exist.

Tuning the constant without cheating

The textbook UCB1 constant explored far too much with hundreds of titles. So I tuned c on the first half of the validation request stream and evaluated on the second half, with different random seeds. The chosen value for the main set (629 titles with at least five ratings in the validation window) was c = 0.2, and the second half confirmed it. That value was then fixed in the test protocol before the test was opened.

What the shelf costs

Reserving 2 of 20 slots means 2 fewer ranked titles on screen. I measured this cost on the main ranking too, so the business trade-off is visible (Chapter 6).

4.6 Putting It Online

The brief asks for serving on AWS Lambda with DynamoDB. Lambda runs code only when a request arrives, with no server to manage, and charges by the millisecond. DynamoDB is a key-value database that answers single lookups in a few milliseconds. Both have free tiers that cover a student project.

What runs where

The Lambda function has a hard size limit: 50 MiB for a zipped upload. TensorFlow and XGBoost do not fit. Even the small CPU build of XGBoost pulls in scipy, which takes the package to 58 MB before adding any data. So I rewrote the scoring parts in plain numpy:

- the two-tower user vector and NCF scoring (a few matrix multiplications), and
- the XGBoost ranker, as a small tree evaluator that reads the trained trees. It matched XGBoost's own predictions on 200,000 real feature rows to within 0.0001 and gave the same top-20 order.

The final package is 49.4 MiB. It contains the code, numpy and the model data, with the content vectors stored at half precision to fit (in a check on 1,500 sampled users, this changed the top 18 for only 2 of them).

A request, step by step

1. The client sends a signed request with a user id. The function URL accepts only requests signed with valid AWS credentials, so it is not open to the public.

2. The function reads the user's record from DynamoDB: the 230 stored candidates, the user's vectors, their rated movies and their latest session.
3. It computes the 37 inputs and scores the candidates with the M2 trees.
4. It fills 18 slots with the best-ranked titles and 2 slots with new releases chosen by UCB (c = 0.2), using shared counts kept in a second DynamoDB table.
5. It returns 20 titles with timing for each step.

In **precomputed mode**, step 2 uses the stored candidate list. In **live mode**, the function builds the candidates fresh with two searches in a Zilliz Cloud vector database, a hosted version of the open-source Milvus (Wang et al., 2021). This is what a platform would use when the catalog changes often. I loaded the two-tower movie vectors and the content vectors into Zilliz's free cluster in Frankfurt, which uses its own automatic index type. Section 5.5 measures how close live results come to the exact precomputed ones.

Keeping it safe and cheap

- All resources live in Frankfurt (eu-central-1), next to the Zilliz cluster, to keep live searches short.
- The deploying user's permissions are limited to resources whose names start with "ott-", in two regions, with explicit blocks on costly services such as EC2.
- The Zilliz key is stored encrypted in AWS Systems Manager, never in the code or in environment variables. No script prints keys, account numbers or addresses, and automated tests check this.
- Logs are kept for 7 days. Tables are billed per request, not reserved in advance.
- Everything was deleted after the measurements, and a teardown check confirmed nothing was left.

4.7 Tools

The models were built in Python 3.12 with TensorFlow and Keras (two-tower, NCF), sentence-transformers (SBERT), XGBoost (ranker), numpy and pandas. The cloud parts used boto3 (AWS) and pymilvus (Zilliz). Every analysis step is a committed script with fixed random seeds, and more than 350 automated tests cover the code. I used Claude Code, an AI coding assistant, throughout the build. Appendix H describes what it did and what I did.

5. Results

5.1 The Five Headline Results

These are the only findings in this report. Each one was written down, with a predicted range, before the test set was opened, and each comes from that single test run on 19,530 users (Section 3.6). Users in the test period liked 14.7 movies each on average.

Table 5.1 *Headline results on the test set, with the predictions committed beforehand*

	Claim	**Predicted**	**Measured (95% interval)**	**Verdict**
P1	M2 ranker beats the two-tower's own order (NDCG@10)	+0.006 to +0.022	**+0.0245** (+0.0229 to +0.0261)	right direction, bigger than predicted
P2	M0 ranker (no context) beats the two-tower's own order (NDCG@10)	+0.005 to +0.020	**+0.0232** (+0.0217 to +0.0247)	right direction, bigger than predicted
P3	Context adds little: M2 minus M0 (NDCG@10)	-0.002 to +0.004	**+0.0013** (+0.0004 to +0.0021)	inside the range
P4	Two-tower retrieves more than item-kNN (Recall@200)	+0.03 to +0.07	**+0.0473** (+0.0441 to +0.0505)	inside the range
P5a	UCB with c = 0.2 beats textbook UCB1 (likes per 1,000 impressions, SIMULATION)	more than +100	**+237.1** (+232.2 to +242.0)	inside the range
P5b	UCB with c = 0.2 against Thompson sampling (SIMULATION)	within plus or minus 30	**+31.9** (+27.9 to +35.9)	just outside the range

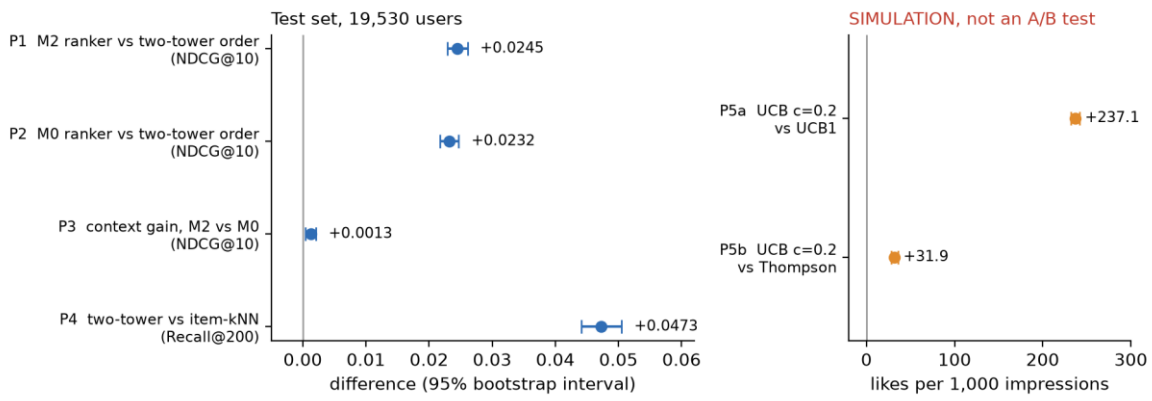


Figure 5.1 *The five headline results with their 95% intervals. Left: test set. Right: bandit simulation, not an A/B test*

In plain words:

- **The learned ranker works.** M2 lifts NDCG@10 from 0.1383 to 0.1628, a relative gain of about 18%. Every one of its inputs was already available to the two-tower model or to NCF, so the gain comes from combining them well.
- **Context adds very little.** The difference between the ranker with and without context is real (the interval excludes zero) but tiny: 0.8% of M0's score.
- **The two-tower model is the better retriever.** It finds 58.5% of a user's future likes in its top 200, against 53.8% for item-kNN, a relative gain of about 9%.
- **A tuned UCB earns far more than the textbook version** in the simulation (807 against 570 simulated likes per 1,000 impressions, about 42% more), and slightly more than Thompson sampling (about 4% more).

Every predicted direction was right. Three of the six size ranges held (P3, P4, P5a). I underestimated the ranker's gain (P1, P2), and the UCB margin over Thompson landed just outside my band (P5b). I report these as they came out.

The sections below give the supporting evidence. Unlike Table 5.1, those numbers are secondary: some intervals exclude zero by chance when many comparisons are made at once, so I read them as context, not as findings.

5.2 Candidate Generation

Figure 5.2 compares four ways of building the candidate pool over the whole catalog of 33,392 movies.

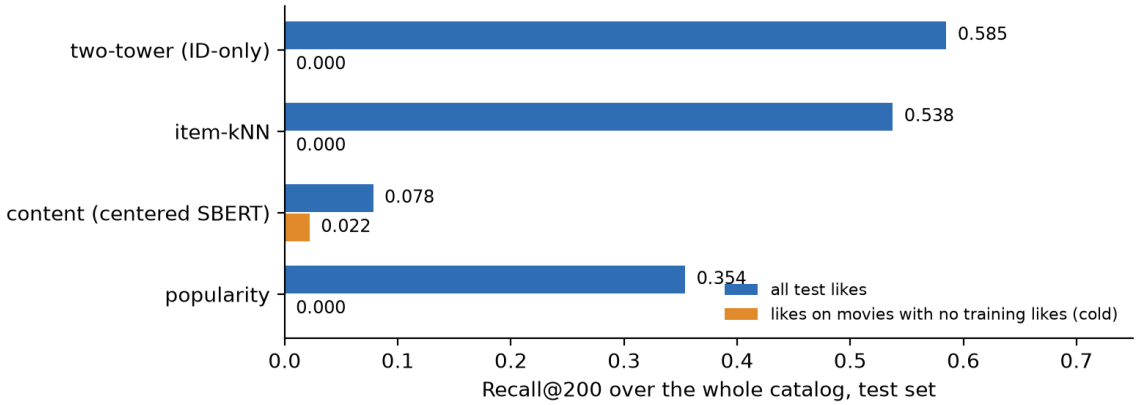


Figure 5.2 *How much of each user's future likes lands in the top 200, test set. Orange bars count only likes on movies that had no likes in training*

The two-tower model leads (Recall@200 of 0.585), followed by item-kNN (0.538) and popularity (0.354). The content model on its own is weak overall (0.078), which is expected: a description tells you less about taste than thousands of other people's ratings do.

However, the orange bars show where content earns its place. For likes on movies that had no ratings in training, every collaborative method scores exactly zero, by construction. Content is the only method that finds any of them (0.022). This is the reason for the separate new-release lane.

The lane's value shows best on the calendar split, where real new releases exist. On validation (360 to 420 users, so the numbers are rough), appending the 30-movie content lane to the 200 two-tower candidates found 10.8% of users' likes on brand-new titles, against 0% for the two-tower model alone, and it did not reduce the hit rate on established titles at all. That 10.8% is partly flattered by the way "new" was defined, since the lane searches the same pool. Under a second definition that the lane was not built around (movies first rated after the cutoff), the figure was 9.5%, which I treat as the safer estimate.

5.3 Ranking

Figure 5.3 shows how different methods order the same 230 candidates on the test set.

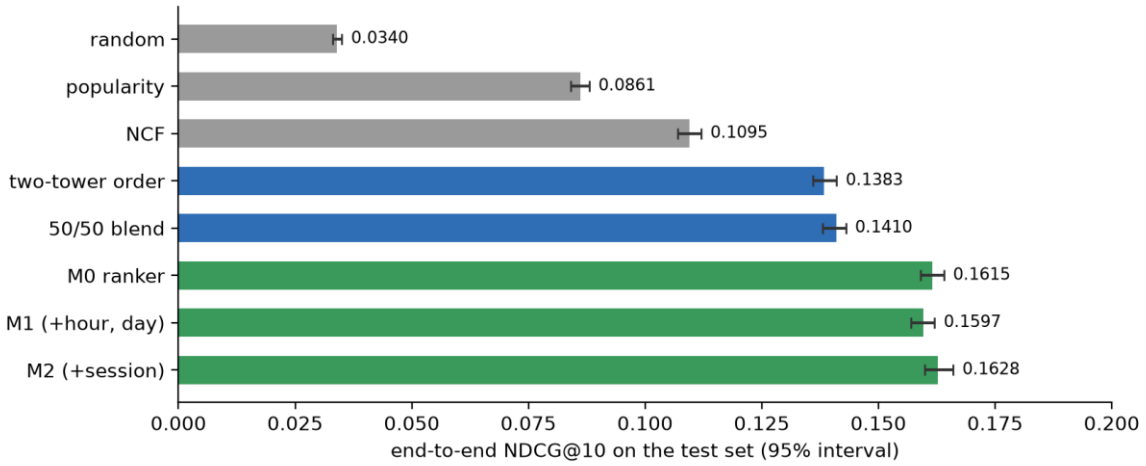


Figure 5.3 *End-to-end NDCG@10 on the test set when the same 230 candidates are ordered by different methods (95% intervals)*

Three things stand out.

First, simple ordering rules hit a ceiling. The two-tower model's own order (0.1383) and a fixed 50/50 blend of two-tower and NCF (0.1410) are close. The trained ranker beats the blend by 0.0218, about 15%. A ranker learns how much to trust each signal for each kind of movie, and a fixed blend cannot.

Second, NCF on its own is weaker than the two-tower order (0.1095 against 0.1383), yet it helps inside the ranker. Its long-term view complements the two-tower model's focus on recent likes. Refreshing NCF's user vectors with the validation-period likes (the fold-in) was worth +0.0027 NDCG@10 to the ranker over stale vectors.

Third, the ranker improves what the user actually sees. Recall@20, the share of future likes that make it onto a 20-title screen, rises from 0.188 with the two-tower order to 0.210 with M2, a relative gain of about 12%.

5.4 Does Context Help?

This was the question I was most curious about, because the brief puts context at the center. The honest answer for this data is: barely.

Table 5.2 *The context ladder on the test set (end-to-end NDCG@10; secondary except P3)*

Model	Inputs	NDCG@10	Change
M0	no context	0.1615	
M1	+ hour, day	0.1597	-0.0019 (-0.0026 to -0.0012)

Model	Inputs	NDCG@10	Change
M2	+ session	0.1628	+0.0032 against M1 (+0.0022 to +0.0041)
M2 against M0	total context (P3)		**+0.0013** (+0.0004 to +0.0021)

Interestingly, hour and day made the test result slightly worse, while the session features helped. On validation, hour and day were also null in the more careful training run, and the session step was small but positive (+0.0018). The pattern is consistent: time of day carries almost no signal here, and session state carries a little.

This makes sense given the data. MovieLens timestamps are UTC, so "evening" is not anyone's real evening. Also, most "sessions" are rating sprees in which people log old favorites, not viewing sessions (Section 3.4). On a real platform with local time and real viewing sessions, I would expect context to matter more. I cannot show that with this data.

Device was tested two ways, both with SIMULATED values:

- A random device with no link to behavior (M3a) added nothing on validation (+0.0001, interval -0.0010 to +0.0013), as it should.
- In the synthetic scenario (**Synthetic device effect, pipeline demonstration, not a finding**), M3b learned the planted rule: +0.0029 on validation (+0.0018 to +0.0041). When the device values were shuffled between users, the gain vanished (+0.0006, interval includes zero). So the pipeline can pick up a device effect when one exists.

The synthetic scenario also says something useful about scale. A fairly strong planted effect (60% of drama-without-comedy likes removed for the 35% of users on SIMULATED mobile) moved NDCG@10 by only about 0.003. A real device effect smaller than that would be hard to see in this kind of offline test.

5.5 New Releases and the Bandit

All numbers in this section come from a SIMULATION, NOT AN A/B TEST.

The bandit simulation on the test window used 2,830 new releases and 43,064 real session starts between March 2017 and November 2019. Figure 5.4 shows the main set: the 761 titles with at least five ratings in the test window (the same rule gave 629 titles in the validation window, where the constant was tuned).

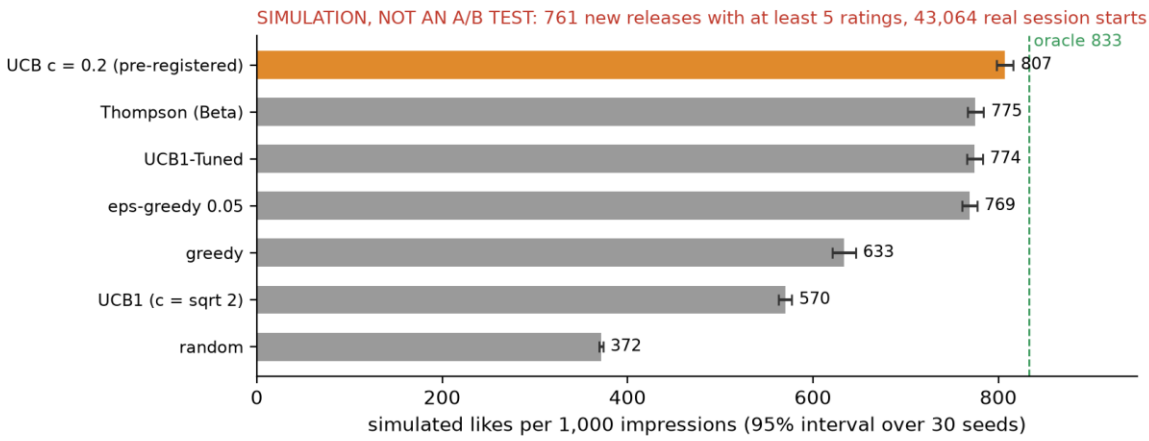


Figure 5.4 *Simulated likes per 1,000 impressions on the 761 titles with at least five ratings, test window (SIMULATION, NOT AN A/B TEST)*

The best possible strategy, one that already knows every title's true like rate, earns 833 likes per 1,000 impressions. Showing titles at random earns 372. Measured against that gap, UCB with c = 0.2 captured 94% of the possible gain, Thompson sampling 87% and textbook UCB1 only 43%. Plain greedy (always show the current best) got stuck early on titles that only looked good.

The same ordering held on the smaller set of 266 titles with at least 20 ratings and on the full set of 2,830 titles with at least one rating. On the larger set, UCB1 did even worse (484, barely above random at 386), because its bonus keeps it exploring hundreds of thin titles.

There is a trade-off hidden in these numbers. UCB with a small constant concentrates impressions on a few titles: its exposure Gini coefficient is 0.98, against 0.90 for Thompson sampling. Every title did get shown at least once, but most impressions went to the leaders. For a platform that also cares about giving each new release a fair chance, Thompson sampling is a reasonable alternative that earns a little less (Chapter 6).

The cost of the shelf. Reserving the last 2 of 20 slots for new releases left NDCG@10 unchanged (the top 10 is untouched) and lowered Recall@20 by 0.0138 (about 6.6% of M2's 0.2100). Putting the two slots inside the top 10 instead cost 0.0175 NDCG@10. The two sides of this trade-off are measured on different scales (simulated likes on new titles against real hits on established ones), so I do not net them into one number.

5.6 Serving in Practice

The whole system was deployed on AWS in Frankfurt and called from my laptop in India. It ran without errors: 400 timed requests, 10 forced cold starts and the demo, with zero failures

and zero live-mode fallbacks. Like the test set, the latency study had its predictions committed before it ran.

Table 5.3 *Measured latency, 200 warm requests per mode (server time is measured inside the function)*

	Precomputed mode	**Live mode**
Server time, median (95% interval)	91.1 ms (85.9 to 96.4)	121.9 ms (121.2 to 123.6)
Server time, 95th / 99th percentile	118.5 / 121.7 ms	144.2 / 163.9 ms
End to end from the laptop, median	338 ms	367 ms
Cold start (5 per mode), median	0.6 s start-up plus 1.5 s for the first request	0.6 s start-up plus 1.8 s for the first request
Peak memory	204 MB of 1,024	220 MB of 1,024

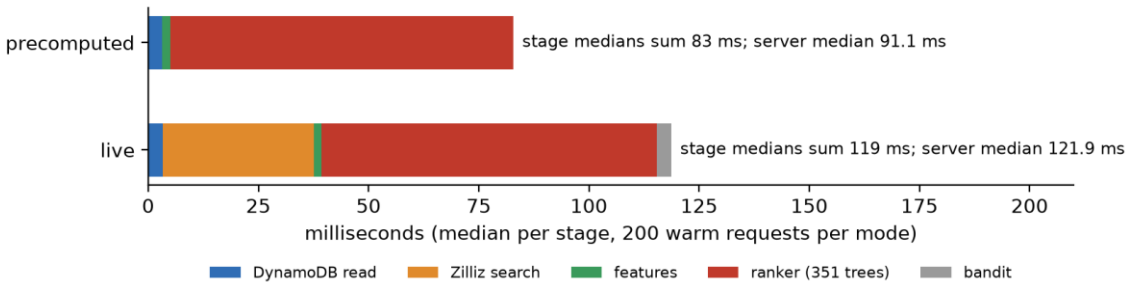


Figure 5.5 *Where the server time goes: median per stage, 200 warm requests per mode*

The biggest surprise was where the time goes. I expected the database and network calls to dominate. Instead, the ranker takes 78 of the 91 ms (85%). Scoring 230 candidates with 351 trees in numpy takes about 11 ms on my laptop, but a Lambda function with 1,024 MB gets well under one full processor core. The live searches add only about 31 ms at the median, and the DynamoDB reads about 3 ms.

The predictions held for 5 of 8 quantities. The three misses:

- I predicted 8 to 30 ms for a warm precomputed request and measured 91 ms. I scaled the laptop timing too optimistically.
- I predicted a cold start of 800 to 3,000 ms and measured 572 ms. However, this was the wrong number to predict. The function loads its data on the first request, not during start-up, so the real wait for a user hitting a cold function is about 2.1 to 2.4 seconds.

- The laptop's network connection setup to Frankfurt took 492 ms, slower than the 100 to 400 ms I expected.

Live search quality. Zilliz's index is approximate: it trades a little accuracy for speed. With its default setting, live search returned 87% of the exact candidates. Raising the search level helped up to level 5, and then stopped improving (Figure 5.6). I chose level 5. At that level, live mode shares about 93% of the top 18 titles with precomputed mode for the same user. Zilliz's documentation suggests that higher levels can reach 99% (Zilliz, n.d.); on the free cluster I could not get past 92%, and I could not find out why.

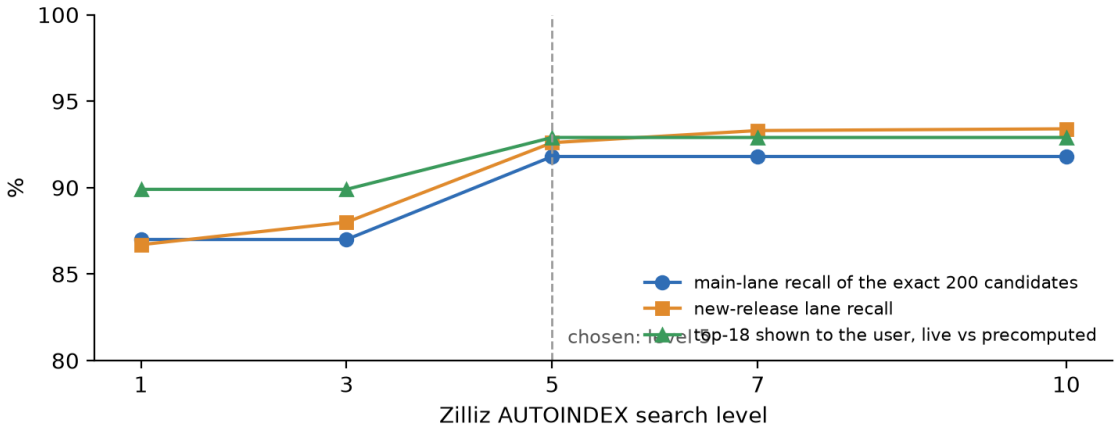


Figure 5.6 *Live search quality by Zilliz search level, 100 users*

One engineering fix is worth mentioning. My first live-search client opened a new secure connection for every search. Reusing one connection cut the search time measured from my laptop from about 960 ms to about 241 ms.

The demo. A short demo page called the deployed function for real users (Appendix F). For one sampled user, moving the request from mid-session to after a three-day break changed 2 of the 18 ranked titles. In the synthetic device scenario, M3b showed on average 5.9 drama-without-comedy titles in the top 18 on a SIMULATED tv and 3.5 on a SIMULATED mobile, across 40 users, while M2 (which ignores device) showed 6.7. That is the planted effect, visible end to end, and it says nothing about real viewers.

5.7 Where I Was Wrong

I want to close this chapter with my misses, because they taught me as much as the hits.

- **The ranker gained more than I predicted** (P1, P2). I expected the two-tower order to be harder to beat.

- **Textbook UCB1 looked like a bad choice at first.** In my first bandit run, plain UCB1 lost to epsilon-greedy and Thompson sampling, and I almost concluded that UCB was the wrong tool. Tuning the constant reversed that verdict.
- **I expected context to matter more.** It did not, on this data.
- **I expected the database to be the slow part of serving.** The ranker was.
- **I assumed a "read-only" Zilliz key would be read-only.** Two keys that I created as read-only could still write on the free cluster (Chapter 7).

6. What This Means for an OTT Business

The results in Chapter 5 are offline numbers on MovieLens. A platform manager would rightly ask what they mean in rupees, hours watched and subscriber churn. I cannot answer that directly without a live test. However, the results do point to clear priorities, and the serving measurements give real cost figures. This chapter covers both.

6.1 The Ranker Is Where the Value Is

The single biggest gain in the project came from the learned ranker. On the test set it improved the quality of the top 10 by about 18% over the candidate generator's own order (P1), and it raised the share of a user's future likes that land on a 20-title screen from 18.8% to 21.0%.

In business terms, that means more of the titles a person will actually enjoy show up where they look first. Gomez-Uribe and Hunt (2015) tie this kind of improvement to fewer cancellations at Netflix, and that is the mechanism I would expect on an Indian platform too: a viewer who finds something good quickly is less likely to leave the app or the subscription.

There is a catch, and it matters for any investment decision. Offline improvements do not always become business improvements (Jannach & Jugovac, 2019). An 18% better offline score is a strong reason to run an online test, not a guarantee of 18% more viewing.

Interestingly, the gain did not come from new data. Every input to the ranker was already available. It came from combining existing signals well. For a platform, that is good news: a learned ranker is a modelling investment, not a data-collection one.

6.2 Do Not Pay for Context Before You Can Measure It

The project brief put context (device, time and session) at the center. On this data, context added less than 1% (P3), and time of day on its own made the test result slightly worse.

I would not conclude that context is useless for Indian OTT. MovieLens has UTC timestamps and rating sprees, not local evenings and viewing sessions, so it is a poor place to find context effects (Section 5.4). On top of this, the synthetic test showed that even a strong planted device effect moves the offline metric only a little. Connected TV is growing fast in India (afaqs!, 2025), and phone and TV viewing may well differ.

My recommendation is to treat context as a hypothesis to test cheaply, not a feature to build first:

- log real local time, device and session data properly;
- add those inputs to the ranker (the pipeline already supports them);
- keep them only if an online test shows a gain.

6.3 Promoting New Releases Has a Price, and It Is Worth Paying

Content is an OTT platform's biggest cost, so new titles need an audience. The project shows both sides of that trade-off.

The cost. Reserving the last 2 of 20 slots for new releases left the top 10 untouched and reduced the share of future likes on screen by 1.4 percentage points (about 6.6% of what the ranker achieves). Putting the new titles inside the top 10 instead hurt the top 10 directly. If a platform wants to promote new releases, the bottom of the first screen is the cheaper place to do it.

The benefit. Without the reserved slots, new titles never reached the ranker's top 20 at all. With them, a tuned UCB strategy captured 94% of the possible gain over picking new titles at random (SIMULATION, NOT AN A/B TEST).

The strategy choice. Tuned UCB earned the most in the simulation but concentrated impressions on a few leading titles. Thompson sampling earned about 4% less but spread exposure more evenly. This becomes a business decision, not a technical one. A platform that mainly wants viewing should prefer tuned UCB. A platform with commitments to producers or content partners (for example, a guaranteed level of visibility for each new release) may prefer Thompson sampling, or tuned UCB with a minimum number of impressions per title.

6.4 What It Costs to Serve

Because the system was actually deployed and measured, I can estimate its running cost from real durations instead of guesses. Table 6.1 uses AWS's published US East prices: $0.20 per million Lambda requests, $0.0000166667 per GB-second of Lambda time, and $0.125 per million read units and $0.625 per million write units for on-demand DynamoDB (Amazon Web Services, n.d.-a, n.d.-b). Frankfurt prices are similar but not identical, so treat these as estimates.

Table 6.1 *Estimated serving cost per million home-screen requests*

Item	Precomputed mode	Live mode	How I estimated it

Item	Precomputed mode	Live mode	How I estimated it
Lambda	about $1.87	about $2.37	1 GB of memory for about 100 ms (precomputed) or 130 ms (live), plus the request fee
DynamoDB	about $1.00	about $1.00	one user record read; for about half the requests, a read of about 10 new-release counters and 2 counter updates
Zilliz vector search	none	not estimated	the free cluster was enough here; a production plan would cost extra
Total	**about $2.90**	**about $3.40 plus Zilliz**	

To put that in context: a platform with one million daily users who each open the home screen three times a day makes about 90 million requests a month. At these rates, serving them in precomputed mode would cost roughly $260 a month (about ₹22,000, assuming ₹85 to the US dollar purely for scale), before logging, data transfer and the offline training runs. Simply put, the serving layer is not where the money goes. Content and marketing dwarf it.

The cost figures also point to two engineering priorities:

- **The ranker is the slow part.** It takes 78 of the 91 ms of a precomputed request (Section 5.6). Giving the function 1,769 MB, which AWS ties to one full processor core (Amazon Web Services, n.d.-c), should cut that time considerably at a similar cost per request, because the higher price per millisecond is offset by fewer milliseconds. Fewer trees would help as well. Both are cheap to try.
- **Cold starts are the bad case for users.** A user who hits a freshly started function waits about 2.1 to 2.4 seconds instead of about 0.1 seconds. Keeping a few instances warm (provisioned concurrency) removes this but costs money around the clock. For a home screen, I would first move the data loading into the start-up phase and measure again.

Precomputed or live? Precomputed mode is exact, faster and cheaper. Live mode costs about 31 ms more and returns about 93% of the same top 18, but it reflects catalog changes immediately, without waiting for the next batch job. A practical design is to serve precomputed candidates by default and use live search only where freshness matters, such as a "new this week" row.

6.5 What a Real Launch Would Measure

Before any rollout, the system should go through an A/B test: a random share of users gets the new recommender, the rest keep the current one, and both groups are compared over the same period (Kohavi et al., 2020). Table 6.2 sketches the metrics I would use.

Table 6.2 *A proposed A/B test plan*

Role	Metric
Primary	Viewing hours per active user per week
Secondary	Share of sessions that start a title from the first row; 30-day retention of subscribers; plays of new releases
Guardrails (must not get worse)	95th-percentile response time; error rate; share of the catalog shown at least once (to catch a popularity trap); complaints and "not interested" actions
Design	Randomize by user, run at least two full weeks to cover weekly patterns, and decide the sample size in advance from the smallest effect worth acting on

The bandit, in particular, needs this. Its numbers in this report come from a simulation, and its real value depends on how often people click a new title, which the simulation does not model.

6.6 Build or Buy?

Managed recommendation services exist, for example Amazon Personalize, and for a platform without a data science team they are a sensible starting point. They are faster to launch and need less specialist staff.

Building in-house, as in this project, makes sense when the recommender is a core part of the product. It allows full control over the ranking logic (such as reserved slots for new releases), the evaluation and the cost. This project shows that the in-house route does not have to be expensive to run. The hard part is the people and the discipline of evaluation, not the servers.

6.7 My Recommendations

If I were advising an Indian OTT platform based on this work, my verdict would be:

1. **Invest in a two-stage system with a learned ranker first.** It delivered the largest and most reliable gain, using data the platform already has.
2. **Test context before building around it.** Log real local time and device data, then let an online test decide.

3. **Reserve the last two slots of the first screen for new releases**, filled by a tuned bandit, and choose between tuned UCB and Thompson sampling based on how much the platform values even exposure across titles.
4. **Serve precomputed recommendations by default**, add live vector search where freshness matters, and fix the ranker's speed before scaling up.
5. **Prove it online.** Run a two-week A/B test with clear guardrails before rolling anything out.

7. Limitations, Risks and Ethics

Like any product review, this one would be incomplete without the weak points. Some of them come from the data, some from the way I evaluated, and some from engineering shortcuts. None of them overturns the main results, but each one limits how far those results can be stretched.

7.1 The Data Is a Stand-In

- **Ratings are not viewing.** Every result predicts what a user will rate 4 stars or more, not what they will click, watch or finish. Most of the "sessions" in MovieLens are rating sprees, not evenings of viewing (Section 3.4).
- **The audience is not India's.** MovieLens users are movie enthusiasts on a research website, rating a largely English-language, Hollywood-heavy catalog. An Indian OTT audience watches series as well as films, in many languages, often on phones and shared family TVs. The methods transfer; the exact numbers do not.
- **Context data is weak.** Timestamps are UTC, and there is no device at all. Every device value in this report is SIMULATED, so the report can say nothing about real device effects.

7.2 The Evaluation Has Limits

- **One test period, known users.** The test set covers each user's next period of activity. It measures how well the system serves people it already knows, not brand-new subscribers.
- **Some calendar leakage remains.** The per-user split keeps each user's own future out of training, but other users' later ratings can still be in training for someone else's test period. The calendar split used for new releases and the bandit does not have this issue.
- **Many comparisons, five findings.** The test run computed many numbers. Only the five pre-registered claims are findings. Some of the secondary differences with intervals that exclude zero may be chance.
- **The bandit is simulated.** Its results assume that each title has a fixed like rate estimated from real ratings, that a shown title is always rated, and that rates do not change over time. Real viewers ignore many recommendations, and tastes shift. SIMULATION, NOT AN A/B TEST.
- **Offline is not online.** None of the results measures viewing time, retention or revenue. Chapter 6 explains what an A/B test would need to show.

7.3 Engineering Shortcuts and Their Risks

- **Live search is approximate.** On the free Zilliz cluster, live search returned about 92% of the exact candidates even at the highest search level, and I could not find out why it stopped there (Section 5.6). Precomputed mode does not have this issue.
- **The "read-only" database key could write.** I created two keys intended to be read-only for the Lambda function. On the free Zilliz cluster, both could still insert data, which my verification script caught. I accepted this as a known limitation for the demo, kept the key separate from the admin key so it can be revoked on its own, stored it encrypted, and limited it to one cluster that held nothing but rebuildable project data. A production system should use a paid tier with properly enforced roles.
- **Cold starts are slow.** A request that lands on a freshly started function waits about 2.1 to 2.4 seconds, because the function loads its data on the first request (Section 5.6).
- **The account is temporary.** The AWS account runs on sign-up credits. If it is on AWS's free plan, it closes when the credits run out or after six months. The deployment was always meant to be torn down after measurement, and it was.

7.4 A Lesson in Cloud Governance

One episode deserves a mention because it is a common real-world trap. My first AWS account was created through a sign-up route that placed it inside an organization I did not control. That organization had a service control policy, a rule set by the organization's owner that overrides any permission inside the account. My own permission tests showed that creating log groups, parameters, DynamoDB tables and Lambda functions was blocked in both regions, even though my own policy allowed all of them.

No amount of fixing on my side could have helped. I closed that account and opened a standalone one, where the same tests passed. The lesson for a business is simple: before planning a cloud project, check who controls the account and what rules sit above it. My permission-test script made the problem visible in two minutes instead of after a failed deployment.

7.5 Ethics and Responsible Recommendation

Privacy. Viewing history reveals a lot about a person: their tastes, beliefs, language and sometimes their health or relationships. In India, the Digital Personal Data Protection Act, 2023 sets rules for how such personal data is collected, used and kept (Ministry of Law and Justice, 2023). This project used only public, anonymous research data. A real deployment

would need consent, a clear purpose, data minimization and limits on how long logs are kept. Even in this project, logs were kept for only 7 days, and the only user data that left the laptop was anonymous MovieLens user numbers with their model vectors and rated-movie lists, all deleted after the measurements.

Popularity bias. Recommenders tend to push already popular titles, because those have the most data. Over time, this can starve the long tail and smaller producers (Abdollahpouri et al., 2017). The reserved new-release slots in this project are one countermeasure, and catalog coverage should be a guardrail metric in any online test (Table 6.2).

Feedback loops. A recommender trained on what people watched, when what they watched was shaped by the recommender, can make everyone's suggestions more alike over time without making anyone happier (Chaney et al., 2018). Exploration, which the bandit provides, and occasional random exposure in the logs both help to break this loop.

Fairness to new titles. The choice between tuned UCB and Thompson sampling (Section 6.3) is also an ethical one. Tuned UCB earns more but gives most impressions to a few titles; Thompson sampling spreads them more evenly. A platform should make that choice deliberately, not by default.

Honest labeling. Simulated and synthetic results can easily be mistaken for real findings, especially in a business setting. That is why every device value, the synthetic M3b scenario and every bandit number in this report carry a label, both in the text and in the demo itself.

7.6 Use of an AI Assistant

I built this project with the help of Claude Code, an AI coding assistant. It wrote most of the code under my direction, ran the local analyses and drafted much of this report in my writing style, which I then reviewed and edited. Many options came from the assistant, but the final decisions on scope, design trade-offs, cloud spending and what to claim were mine, and every cloud action was run by me. Appendix H gives the full statement. I mention it here because it is a real factor in how the work was done, and the reader should know it.

8. Conclusion and Future Work

8.1 Answers to the Research Questions

RQ1. Does a learned ranker beat the candidate generator's own order? Yes, clearly. On the test set, the M2 ranker improved NDCG@10 by +0.0245 (about 18%), and the ranker without context by +0.0232. Both gains were larger than I predicted.

RQ2. Does context help? Only a little, on this data. Time and session together added +0.0013 NDCG@10 (under 1%); time of day on its own slightly hurt. The synthetic device test showed that the pipeline can learn a device effect when one exists, but MovieLens holds no real device or local time, so real context effects remain untested.

RQ3. Does the two-tower model retrieve better than item-based nearest neighbors? Yes. It found 58.5% of users' future likes in its top 200, against 53.8% for item-kNN, a difference of +0.0473 that matched my prediction.

RQ4. Which bandit strategy works best for new releases? In the simulation, UCB with a tuned constant (c = 0.2) captured 94% of the possible gain, beating textbook UCB1 by 237 simulated likes per 1,000 impressions and Thompson sampling by 32. Tuning mattered more than the choice of algorithm: untuned UCB1 was the worst of the learning strategies, behind even plain greedy. These are SIMULATION results, NOT an A/B test.

RQ5. Can a content lane surface titles the other methods cannot see? Yes. On movies with no ratings in training, every collaborative method scored exactly zero, while the content model found some of them. A 30-title content lane appended to the candidate list found about 10% of users' likes on brand-new titles without hurting the rest of the list.

RQ6. Can it run cheaply and fast on serverless infrastructure? Yes, with one bottleneck. The deployed system answered warm requests in 91 ms (precomputed) and 122 ms (live) of server time, with zero errors across 400 timed requests, at an estimated $2.90 to $3.40 per million requests. The ranker takes most of that time, and cold starts take about 2 seconds.

8.2 What I Take Away

Looking back, three lessons stand out for me.

First, **the biggest gains came from combining signals, not from collecting new ones.** The ranker added about 18% using inputs the system already had. Context, which sounded like the exciting part, added almost nothing here.

Second, **tuning beat cleverness.** The textbook bandit looked like a poor choice until its one constant was tuned on held-out data. The same was true of small details like centering the text embeddings (a five-fold gain) and the popularity correction in the two-tower model (the difference between a working model and a broken one).

Third, **being strict about evaluation was worth the effort.** Writing predictions down first, comparing every model with simple baselines and looking at the test set only once made the results slower to get but much easier to trust. It also made my mistakes visible, which is the point.

8.3 Future Work

If I had more time, data or budget, these are the things I would do next, roughly in order of value:

1. **Run an online A/B test** on a real platform or a beta group, with viewing hours as the main metric and the guardrails from Table 6.2. Nothing else would tell as much.
2. **Use real viewing logs with local time and device**, then re-run the context ladder. This is the only way to answer RQ2 properly.
3. **Make the ranker faster:** give the function one full processor core, try fewer trees, and move data loading into the start-up phase to shorten cold starts.
4. **Add Indian-language content.** A multilingual text model would let the content lane handle Hindi, Tamil, Telugu and other titles, which matters far more for an Indian platform than for MovieLens.
5. **Treat fairness of exposure as a goal**, for example a bandit that guarantees each new release a minimum number of impressions, and measure the cost of that guarantee.
6. **Fix the approximate-search gap** by testing other index settings or a paid vector database tier, and confirm whether live search can reach the 99% recall the documentation suggests.

8.4 Closing

The brief asked for a hyper-personalized OTT recommendation engine with four specific parts. I built all four, connected them into one working service, deployed it on AWS, measured it and took it down again. The learned ranker delivered a clear gain, the content lane and the bandit gave new releases a path to the screen, and the whole thing ran for a few dollars per million requests.

To be fair, it is a strong prototype, not a finished product. It still needs real viewing data and an online test before anyone should bet a business on it. Still, every number in this report can be traced to a committed script and a saved result, and every claim was checked before it was made. For a one-month project, I am happy with that.

References

Abdollahpouri, H., Burke, R., & Mobasher, B. (2017). Controlling popularity bias in learning-to-rank recommendation. In *Proceedings of the Eleventh ACM Conference on Recommender Systems* (pp. 42-46). ACM. https://doi.org/10.1145/3109859.3109912

Adomavicius, G., & Tuzhilin, A. (2011). Context-aware recommender systems. In F. Ricci, L. Rokach, B. Shapira, & P. B. Kantor (Eds.), *Recommender systems handbook* (pp. 217-253). Springer. https://doi.org/10.1007/978-0-387-85820-3_7

afaqs! news bureau. (2025, September 16). *India's OTT audience hits 601 million, paid subscriptions at 148 million: Ormax report*. afaqs! https://afaqs.com/news/media/indias-ott-audience-hits-601-million-paid-subscriptions-at-148-million-ormax-report-10469442

Amazon Web Services. (n.d.-a). *Amazon DynamoDB pricing for on-demand capacity*. Retrieved October 27, 2026, from https://aws.amazon.com/dynamodb/pricing/on-demand/

Amazon Web Services. (n.d.-b). *AWS Lambda pricing*. Retrieved October 27, 2026, from https://aws.amazon.com/lambda/pricing/

Amazon Web Services. (n.d.-c). *Lambda quotas*. Retrieved October 26, 2026, from https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html

Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning, 47*(2-3), 235-256. https://doi.org/10.1023/A:1013689704352

Burges, C. J. C. (2010). *From RankNet to LambdaRank to LambdaMART: An overview* (Technical Report MSR-TR-2010-82). Microsoft Research.

Chaney, A. J. B., Stewart, B. M., & Engelhardt, B. E. (2018). How algorithmic confounding in recommendation systems increases homogeneity and decreases utility. In *Proceedings of the 12th ACM Conference on Recommender Systems* (pp. 224-232). ACM. https://doi.org/10.1145/3240323.3240370

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794). ACM. https://doi.org/10.1145/2939672.2939785

Covington, P., Adams, J., & Sargin, E. (2016). Deep neural networks for YouTube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems* (pp. 191-198). ACM. https://doi.org/10.1145/2959100.2959190

Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.

Ferrari Dacrema, M., Cremonesi, P., & Jannach, D. (2019). Are we really making much progress? A worrying analysis of recent neural recommendation approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems* (pp. 101-109). ACM. https://doi.org/10.1145/3298689.3347058

Gomez-Uribe, C. A., & Hunt, N. (2015). The Netflix recommender system: Algorithms, business value, and innovation. *ACM Transactions on Management Information Systems, 6*(4), Article 13. https://doi.org/10.1145/2843948

Harper, F. M., & Konstan, J. A. (2015). The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems, 5*(4), Article 19. https://doi.org/10.1145/2827872

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web* (pp. 173-182). https://doi.org/10.1145/3038912.3052569

Jannach, D., & Jugovac, M. (2019). Measuring the business value of recommender systems. *ACM Transactions on Management Information Systems, 10*(4), Article 16. https://doi.org/10.1145/3370082

Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems, 20*(4), 422-446. https://doi.org/10.1145/582415.582418

Kohavi, R., Tang, D., & Xu, Y. (2020). *Trustworthy online controlled experiments: A practical guide to A/B testing*. Cambridge University Press. https://doi.org/10.1017/9781108653985

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer, 42*(8), 30-37. https://doi.org/10.1109/MC.2009.263

Lattimore, T., & Szepesvári, C. (2020). *Bandit algorithms*. Cambridge University Press. https://doi.org/10.1017/9781108571401

Li, L., Chu, W., Langford, J., & Schapire, R. E. (2010). A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web* (pp. 661-670). https://doi.org/10.1145/1772690.1772758

Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 42*(4), 824-836. https://doi.org/10.1109/TPAMI.2018.2889473

Ministry of Law and Justice, Government of India. (2023). *The Digital Personal Data Protection Act, 2023* (Act No. 22 of 2023).

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing* (pp. 3980-3990). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1410

Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web* (pp. 285-295). https://doi.org/10.1145/371920.372071

Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika, 25*(3/4), 285-294. https://doi.org/10.2307/2332286

Wang, J., Yi, X., Guo, R., Jin, H., Xu, P., Li, S., Wang, X., Guo, X., Li, C., Xu, X., Yu, K., Yuan, Y., Zou, Y., Long, J., Cai, Y., Li, Z., Zhang, Z., Mo, Y., Gu, J., ... Xie, C. (2021). Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data* (pp. 2614-2627). ACM. https://doi.org/10.1145/3448016.3457550

Yi, X., Yang, J., Hong, L., Cheng, D. Z., Heldt, L., Kumthekar, A., Zhao, Z., Wei, L., & Chi, E. (2019). Sampling-bias-corrected neural modeling for large corpus item recommendations. In *Proceedings of the 13th ACM Conference on Recommender Systems* (pp. 269-277). ACM. https://doi.org/10.1145/3298689.3346996

Zilliz. (n.d.). *Tune recall rate*. Zilliz Cloud documentation. Retrieved October 27, 2026, from https://docs.zilliz.com/docs/tune-recall-rate

Appendix A. Reproducibility

Every number in this report comes from a committed script and a saved result file in the project repository ([repository link]). The project history holds more than 80 commits, and about 360 automated tests cover the code.

A.1 Environment

- Python 3.12 in a virtual environment managed with uv; exact package versions in `requirements.lock.txt`.
- Main libraries: TensorFlow 2.20 with Keras 3, sentence-transformers, XGBoost, numpy, pandas, boto3, pymilvus.
- Hardware: a laptop with 16 GB of RAM (about 9 GB usable under WSL2) and no GPU. A memory guard script was set to stop any run above 7.5 GB.

A.2 Data

- MovieLens 25M, downloaded by `scripts/download_movielens.py`; a fixed random sample of 20,000 users (seed 42) taken by the shared data module.
- TMDB descriptions and release dates fetched by `scripts/fetch_tmdb.py` with the project's own API key (stored outside the repository). TMDB data is not redistributed.

A.3 Scripts by Module

Table A.1 *Where each part of the work lives*

Module	What it does	Main scripts
1 and 2	Data audit, time splits, test lock, evaluation code, baselines	`run_module1.py`, `build_user_split.py`, `run_module2.py`
3	TMDB fetch, Sentence-BERT embeddings, content retrieval	`fetch_tmdb.py`, `build_embeddings.py`, `run_module3.py`
4 and 4b	Two-tower training, candidate mixes, exports for the vector database	`train_two_tower.py`, `run_module4_all.py`, `run_module4b.py`
5	NCF training and the 230-slot candidate lists	`build_candidates.py`, `train_ncf.py`, `run_module5_all.py`
6	Ranker inputs, XGBoost ranker, context ladder	`build_ranker_features.py`, `run_module6_all.py`
7 and 7b	Bandit simulation and tuning	`run_module7_all.py`, `run_module7b.py`

Module	What it does	Main scripts
8	Frozen artifacts and the one-shot test run	`prepare_test.py`, `unlock_test.py`
9	Serving: bundle, Lambda zip, deployment, Zilliz, latency, demo, teardown	`build_serving_artifacts.py`, `build_lambda_zip.py`, `load_zilliz.py`, `verify_zilliz.py`, `latency_study.py`, `demo.py`, `aws_teardown.py`, `zilliz_teardown.py`
Report	Figures and this document	`make_report_figures.py`, `build_report_docx.py`, `check_report_style.py`

A.4 The One-Shot Test Run

- The test portion was locked in code during development: reading it raised an error, and automated scans checked that no tuning code referred to it.
- The protocol (`reports/module8_protocol.md`) and predictions (`reports/module8_predictions.md`) were committed in commit e3d5da3 before the test was opened.
- Before the run, a provenance check re-ran the committed preparation script from a clean copy of the repository: all 8 compared artifacts came out byte-identical to the frozen ones.
- The test-opening script ran once, on 26 October 2026 from 16:08 to 16:17, on code commit cc20239, with 12 frozen artifacts checked against their recorded fingerprints. Its log (`reports/TEST_UNLOCK_LOG.json`) records status "done" and one attempt. The script refuses to run again because its own log exists in the project history.
- The results file (`reports/module8_test_results.txt`) was committed unedited; its SHA-256 fingerprint begins dd475cea.

A.5 Serving Measurements

- Latency predictions were committed in commit 519e241 before the latency study ran; the study script refuses to start if that file has changed.
- Raw latency results: `reports/module9_latency_results.json`. Demo page: `reports/module9_demo.html`.
- The level study of the vector database (Figure 5.6) was run once from the laptop; its numbers are recorded in the progress log.

Appendix B. Full Result Tables

All tables in this appendix are secondary results, except where marked as a headline claim (P1 to P5). End-to-end (e2e) metrics count every liked movie; reachable (rr) metrics count only likes that were inside the 230 candidates.

Table B.1 *Ordering the 230 candidates, test set, 19,530 users (mean, 95% interval in brackets)*

Method	e2e NDCG@10	e2e Recall@20	rr NDCG@10	rr Recall@20
Random order	0.0340 [0.033, 0.035]	0.0553	0.0413	0.0950
Popularity	0.0861 [0.084, 0.088]	0.1110	0.1042	0.1902
Two-tower's own order	0.1383 [0.136, 0.141]	0.1880	0.1630	0.2836
NCF, fold-in	0.1095 [0.107, 0.112]	0.1535	0.1316	0.2492
NCF, stale	0.1020 [0.100, 0.104]	0.1430	0.1227	0.2338
50/50 blend, fold-in	0.1410 [0.138, 0.143]	0.1888	0.1668	0.2944
50/50 blend, stale	0.1353 [0.133, 0.138]	0.1828	0.1599	0.2848
M0 ranker, no context	0.1615 [0.159, 0.164]	0.2068	0.1917	0.3256
M1 ranker, + hour and day	0.1597 [0.157, 0.162]	0.2054	0.1893	0.3216
M2 ranker, + session (primary)	**0.1628 [0.160, 0.166]**	**0.2100**	**0.1937**	**0.3315**
M2 with stale NCF	0.1602 [0.157, 0.163]	0.2055	0.1906	0.3245
Oracle (perfect order)	0.7765	0.5733	1.0000	0.9914

Table B.2 *Retrieval over the whole catalog, test set, 19,530 users*

Method	Recall@10	Recall@50	Recall@200	NDCG@10	Recall@200, established titles	Recall@200, cold titles
Two-tower (ID only)	0.1193	0.3174	**0.5850**	0.1383	0.5856	0.0000
Item-kNN	0.0993	0.2759	0.5377	0.1258	0.5382	0.0000

Method	Recall@10	Recall@50	Recall@200	NDCG@10	Recall@200, established titles	Recall@200, cold titles
Content (centered SBERT)	0.0103	0.0310	0.0783	0.0125	0.0783	0.0221
Popularity	0.0555	0.1634	0.3541	0.0743	0.3545	0.0000

Table B.3 *Bandit simulation on the test window: simulated likes per 1,000 impressions (SIMULATION, NOT AN A/B TEST)*

Strategy	266 titles (20+ ratings)	761 titles (5+ ratings, main set)	2,830 titles (1+ rating)
Random	411.3	371.6	385.9
Greedy	619.9	633.3	644.1
Epsilon-greedy (0.05)	707.8	769.0	844.9
UCB1 (textbook constant)	623.8	570.2	484.2
UCB, c = 0.2 (pre-registered)	**727.1**	**807.3**	**871.6**
UCB1-Tuned	722.0	774.5	747.9
Thompson sampling	720.4	775.3	786.6
Oracle (knows the true rates)	748.7	833.2	921.0

On the main set, the exposure Gini coefficient (0 = every title shown equally, 1 = one title gets everything) was 0.15 for random, 0.53 for UCB1, 0.90 for Thompson sampling, 0.91 for UCB1-Tuned, 0.95 for epsilon-greedy, 0.98 for UCB with c = 0.2 and 0.99 for greedy. Greedy never showed 88% of the titles; UCB with c = 0.2 showed every title at least once.

Table B.4 *Cost of reserving slots for new releases, M2 ranking, test set (difference against no reservation)*

Reserved slots	NDCG@10	Recall@20
2 at the end (positions 19, 20)	0.0000	-0.0138 [-0.0146, -0.0130]
3 at the end (positions 18 to 20)	0.0000	-0.0208 [-0.0217, -0.0198]
2 inside the top 10 (positions 5, 10)	-0.0175 [-0.0180, -0.0170]	-0.0138 [-0.0146, -0.0130]

Table B.5 *Latency of the deployed service, 200 warm requests per mode (milliseconds)*

	Precomputed	**Live**
Server time: median [95% interval]	91.1 [85.9, 96.4]	121.9 [121.2, 123.6]
Server time: 95th percentile [95% interval]	118.5 [108.6, 120.7]	144.2 [141.1, 157.3]
Server time: 99th percentile	121.7	163.9
Laptop end to end: median / 95th / 99th	338 / 361 / 382	367 / 400 / 437
Stage medians: DynamoDB read / search / inputs / ranker / bandit	3.2 / 0.0 / 1.8 / 77.8 / 0.1	3.2 / 34.4 / 1.6 / 76.1 / 3.3
Cold start (5 per mode): start-up / first request, medians	583 / 1,487	561 / 1,786
Peak memory	204 MB	220 MB

Table B.6 *Live search quality by Zilliz search level, 100 users*

Level	**Main-lane recall of the exact candidates**	**New-release lane recall**	**Top-18 overlap, live against precomputed**	**Same top-18 set**
1 (default)	87.0%	86.7%	89.9%	13 of 100
3	87.0%	88.0%	89.9%	13 of 100
5 (chosen)	**91.8%**	**92.6%**	**92.9%**	**24 of 100**
7	91.8%	93.3%	92.9%	24 of 100
10	91.8%	93.4%	92.9%	24 of 100

Table B.7 *The context ladder on validation, 15,530 held-out users, out-of-fold scores (supporting evidence)*

Method	**e2e NDCG@10**	**e2e Recall@20**
Two-tower's own order	0.1242	0.2321
NCF order	0.0899	0.1836
50/50 blend	0.1233	0.2340
M0 ranker	0.1370	0.2508
M1 ranker	0.1379	0.2512
M2 ranker	0.1388	0.2529
M3a ranker (+ SIMULATED independent device)	0.1389	0.2522

Appendix C. Predictions Made Before the Results

From Module 4 onward, I committed my expected results to the project history before running anything. Table C.1 lists those files. The detailed predictions-versus-actuals for each module are in the module reports in the repository; the two that matter most for this report are reproduced below.

Table C.1 *Pre-registration files and the commits that fixed them*

Module	File	Commit	Date
4: two-tower model	`reports/module4_predictions.md`	58f485e	26 Oct 2026
5: NCF	`reports/module5_predictions.md`	9b46c62	26 Oct 2026
6: ranker and context	`reports/module6_predictions.md`	eb9bad9	26 Oct 2026
7: bandit	`reports/module7_predictions.md`	4e98d3b	26 Oct 2026
7b: bandit tuning	`reports/module7b_predictions.md`	def2b66	26 Oct 2026
8: test protocol and predictions	`reports/module8_protocol.md`, `reports/module8_predictions.md`	e3d5da3	26 Oct 2026
9: serving latency	`reports/module9_latency_predictions.md`	519e241	27 Oct 2026

Table C.2 *The headline test claims: prediction against result*

Claim	Predicted (verbatim range)	Result	Verdict
P1: M2 against the two-tower order, NDCG@10	positive (80%), +0.006 to +0.022	+0.0245 [+0.0229, +0.0261]	direction right, above the range
P2: M0 against the two-tower order, NDCG@10	positive (80%), +0.005 to +0.020	+0.0232 [+0.0217, +0.0247]	direction right, above the range
P3: M2 against M0, NDCG@10	between -0.002 and +0.004 (70%)	+0.0013 [+0.0004, +0.0021]	inside the range
P4: two-tower against item-kNN, Recall@200	positive (85%), +0.03 to +0.07	+0.0473 [+0.0441, +0.0505]	inside the range
P5a: UCB c = 0.2 against UCB1 (SIMULATION)	better (90%), by more than +100 per 1,000 (80%)	+237.1 [+232.2, +242.0]	right on both parts
P5b: UCB c = 0.2 against Thompson (SIMULATION)	within plus or minus 30 per 1,000 (65%)	+31.9 [+27.9, +35.9]	just outside the range

The eight secondary test predictions came out as 7 right and 1 not evaluable (the test script did not report the share of candidates removed by the release-date filter, which one prediction was about).

Table C.3 *Serving latency: prediction against result*

Id	Quantity	Predicted	Measured	Verdict
L1	Warm server time, precomputed, median	8 to 30 ms	91.1 ms	miss
L2	Warm server time, live, median	25 to 150 ms	121.9 ms	inside
L3	Live minus precomputed, medians	15 to 130 ms	30.7 ms	inside
L4	Laptop end to end, precomputed, median	150 to 500 ms	338.4 ms	inside
L5	Cold start (start-up phase), median	800 to 3,000 ms	572.1 ms	miss
L6	Peak memory	150 to 450 MB	220 MB	inside
L7	Laptop connection setup to Frankfurt, median	100 to 400 ms	492.2 ms	miss
L8	Live-mode fallbacks during the warm run	0	0	inside

The latency predictions file also records that they were written after I had seen the smoke-test timings, so they were informed guesses, not blind ones.

Appendix D. Security and Access Design

The policies are kept as JSON files in the repository (`infra/iam/`) with the account number replaced by a placeholder. Automated tests check them for the rules below.

D.1 Who Can Do What

Table D.1 *The identities and their permissions*

Identity	Used for	Allowed	Explicitly blocked
IAM user `ott-dev` (managed policy `ott-dev-serving`)	Running every deployment, test and teardown script from the laptop	Create, use and delete DynamoDB tables, Lambda functions, log groups and parameters whose names start with "ott", in Frankfurt and Mumbai only; pass the one Lambda role to Lambda; list resource names for the teardown check	Creating, attaching or changing any IAM role or policy; EC2; provisioned concurrency; DynamoDB backups and global tables; any other region
Role `ott-recsys-lambda-role`	The Lambda function at run time	Write its own logs; read user records; read and update the bandit counters; read the one Zilliz parameter	Everything else
Zilliz key (stored in AWS Systems Manager)	Live search from the function	Search the three project collections (it could also write; see Section 7.3)	Nothing else in the project used it

The `ott-dev` policy is about 2,730 characters long, above the 2,048-character limit for policies attached directly to a user, so it is a customer managed policy (limit 6,144 characters).

D.2 How Secrets Were Handled

- No key, password, account number or private address appears in the code, the reports or the project history. Secrets live in a local `.env` file that is excluded from version control, and the coding assistant was blocked from reading that file and the AWS credential folder.
- The Zilliz key reaches the Lambda function only through one encrypted parameter in AWS Systems Manager, read at start-up and kept in memory. The function has no environment variables holding secrets.
- Every script prints only words and counts. Error messages show the type of error, never its text, because error text can contain addresses or keys. Automated tests feed fake

"canary" secrets through every failure path and check that they never appear in any output.

- The function URL accepts only requests signed with AWS credentials; it is not open to the public.

D.3 Safe Teardown

- Two scripts remove everything the project created: one for AWS (`aws_teardown.py`) and one for Zilliz (`zilliz_teardown.py`). Both default to a dry run that only reports, need a typed confirmation word to delete, touch only the project's own names, and re-check afterwards.
- A check that cannot confirm a resource is gone (for example, because of a permission error) is reported as "unverified", never as "absent".
- After the final measurements, the teardown removed all AWS resources in both regions and all three Zilliz collections, and a final dry run confirmed nothing was left.

Appendix E. Deployment Runbook (Summary)

The full runbook is `reports/module9_runbook.md` in the repository. Every command below was run by me from the laptop, in this order, with the IAM user's profile. Each step prints only words and counts.

Table E.1 *Deployment steps*

Step	What happens	Script
1. Account set-up (by hand)	Root MFA, budget alert, Lambda role, managed policy, IAM user and its key, local profile	(console; checklist in `reports/module9_account_setup.md`)
2. Permission test	Creates, uses and deletes tiny test resources in Frankfurt and Mumbai; reports ALLOWED or DENIED per action	`aws_permtest.py`
3. Vector database	Checks the exports, creates the three Zilliz collections and loads 49,843 rows (16,450 two-tower and 33,392 content vectors, plus one test row)	`load_zilliz.py`
4. Verification	Searches with the function's key through two client libraries; optional level study	`verify_zilliz.py`
5. DynamoDB	Creates two on-demand tables and loads 20,000 user records; reads 40 back	`cloudshell/load_dynamodb.py`
6. Zilliz key	Stores the key in one encrypted parameter	`put_ssm_secrets.py`
7. Lambda	Creates the log group (7-day retention), uploads the 49.4 MiB package, creates the signed-only function URL	`cloudshell/deploy_lambda.py`
8. Smoke test	Health check, precomputed and synthetic-device requests, and a live request	`cloudshell/deploy_lambda.py`
9. Measurements	Latency study, then the demo page	`latency_study.py`, `demo.py`
10. Teardown	Removes every AWS resource and Zilliz collection, then confirms	`aws_teardown.py`, `zilliz_teardown.py`

A fallback route through AWS CloudShell exists in the runbook for accounts where the laptop route is blocked by an organization's policy (Section 7.4); it was not needed for the final deployment.

Appendix F. The Demo

After the latency study, a script called the deployed function for real users and wrote a demo page (`reports/module9_demo.html`). Every list on the page came from a live request to the function in Frankfurt. The page has four sections:

- **A.** The real M2 ranker for one user, mid-session and after a three-day break (2 of 18 titles changed).
- **B.** The synthetic device scenario, the same user with a SIMULATED tv and a SIMULATED mobile (Figure F.1).
- **C.** Live vector search against precomputed candidates for the same user (16 of 18 titles shared).
- **D.** The new-release slots under simulated feedback (200 requests, 145 new-release impressions, 40% simulated likes; SIMULATION, NOT AN A/B TEST).

Every section carries its label on the page itself, so a screenshot cannot lose it.

Synthetic device effect, pipeline demonstration, not a finding

SIMULATED device = tv
12 of 18 are Drama-without-Comedy

1	Fight Club (1999)
2	Shawshank Redemption, The (1994)
3	Total Recall (1990)
4	Kill Bill: Vol. 1 (2003)
5	Rock, The (1996)
6	Déjà Vu (Deja Vu) (2006)
7	Bourne Supremacy, The (2004)
8	Lucky Number Slevin (2006)
9	Kill Bill: Vol. 2 (2004)
10	Apocalypto (2006)
11	Jurassic Park (1993)
12	The Devil's Advocate (1997)
13	Catch Me If You Can (2002)
14	American History X (1998)
15	Donnie Darko (2001)
16	Green Mile, The (1999)
17	Beautiful Mind, A (2001)
18	American Beauty (1999)

SIMULATED device = mobile
7 of 18 are Drama-without-Comedy

1	Kill Bill: Vol. 1 (2003)
2	Total Recall (1990)
3	Rock, The (1996)
4	Shawshank Redemption, The (1994)
5	Bourne Supremacy, The (2004)
6	Déjà Vu (Deja Vu) (2006)
7	Fight Club (1999)
8	Jurassic Park (1993)
9	Fargo (1996)
10	The Devil's Advocate (1997)
11	Vanilla Sky (2001)
12	Cloverfield (2008)
13	Lucky Number Slevin (2006)
14	Kill Bill: Vol. 2 (2004)
15	Terminal, The (2004)
16	Apocalypto (2006)
17	Catch Me If You Can (2002)
18	Casino Royale (2006)

Shaded rows: Drama-without-Comedy titles. One sampled user shown (MovieLens titles).
Average over 40 users: M3b tv 5.92, M3b mobile 3.50, M2 (ignores device) 6.67.

Figure F.1 *Section B of the demo page, redrawn from the saved page: the same user's top 18 under a SIMULATED tv and a SIMULATED mobile. The synthetic rule removed drama-without-comedy likes on mobile during training*

Appendix G. Glossary for Non-Technical Readers

Table G.1 *Terms used in this report*

Term	Plain meaning
A/B test	Showing two versions to randomly split groups of real users and comparing what they do
Bandit (multi-armed)	A method that decides what to show next while still learning which options are good, balancing trying new options with using proven ones
Baseline	A simple method that a new model must beat to be worth its complexity
Bootstrap interval	A range for a result, found by resampling the users many times; if it excludes zero, the difference is unlikely to be luck
Candidate generation	The first, fast step that narrows thousands of titles to a few hundred for each user
Cold start	The difficulty of recommending a title (or to a user) with no history yet
Cold start (serverless)	The extra delay when a cloud function has to start up before answering its first request
Collaborative filtering	Recommending from what similar people liked, without knowing what the items are about
Content-based filtering	Recommending from what the items are (genre, description)
Embedding	A list of numbers that represents a movie, a user or a text, so that similar things get similar numbers
Fold-in	Updating a user's vector with new likes without retraining the whole model
Gini coefficient (exposure)	How unevenly impressions are spread across titles: 0 means equal, 1 means one title gets everything
Lambda (AWS)	A cloud service that runs code only when a request arrives and charges by the millisecond
NDCG@10	A score for the top 10 of a list that gives more credit for liked titles near the top
Out-of-fold	Scoring each user with a model that was trained without that user, to avoid flattering results
Pre-registration	Writing down predictions before running an experiment
Ranker	The second step that orders the candidates using many signals
Recall@K	The share of a user's future likes that appear in their top K
Service control policy	A rule set by an organization's owner that overrides permissions inside its member accounts

Term	Plain meaning
Two-tower model	A neural network with one part for users and one for items, trained so that a user's vector points toward items they will like
UCB	Upper Confidence Bound, a bandit method that adds an optimism bonus to options that have not been tried much
Vector database	A database that quickly finds the stored vectors closest to a given one

Appendix H. Tools Used and AI Assistance

H.1 Software and Services

- **Modelling:** Python 3.12, TensorFlow 2.20 and Keras 3 (two-tower model, NCF), sentence-transformers (Sentence-BERT), XGBoost (ranker), numpy and pandas.
- **Data:** MovieLens 25M (GroupLens Research) and The Movie Database (TMDB) API.
- **Cloud:** AWS Lambda, DynamoDB, Systems Manager Parameter Store, CloudWatch Logs and IAM, in eu-central-1 (Frankfurt); Zilliz Cloud free cluster (hosted Milvus), also in Frankfurt.
- **Report:** matplotlib for the figures and python-docx to assemble this document, both driven by scripts in the repository.

H.2 Use of an AI Assistant

I used Claude Code (Anthropic), an AI coding assistant, throughout this project.

What the assistant did. Under my direction, it wrote most of the code and the automated tests, ran the analyses on my laptop, proposed designs and options at each step, drafted the module reports and the progress log, and drafted this report in my writing style from my earlier published articles, which I then reviewed and edited.

What I did. I set the scope and the order of work, chose between the options at every decision point (for example, the evaluation rules, the cloud region, how to handle the database key, and which results to treat as findings), approved every step that could cost money, created and controlled the AWS and Zilliz accounts, and ran every command that touched the cloud myself. I reviewed the results and the text and take responsibility for the claims in this report.

Safeguards. The assistant was blocked from reading my secrets file and my AWS credentials. Every result in this report comes from a committed script and a saved file, so the numbers can be checked independently of the assistant. The text was checked by a script for typical AI phrasing and against the saved results for every number.